

第3章

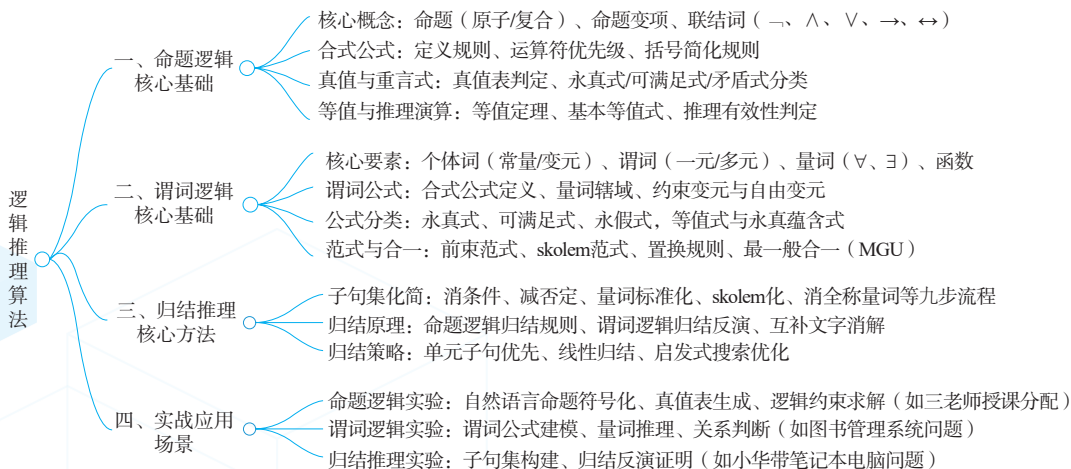
逻辑推理算法

知识导图

视频



逻辑推理
算法



引言

在掌握搜索算法的问题求解逻辑后，本章将聚焦人工智能的核心推理范式——逻辑推理，这是实现机器“理性思考”的关键技术支撑。逻辑推理作为符号主义人工智能的核心方法，通过形式化的逻辑语言刻画知识、构建规则，让机器能够基于严格的逻辑规则推导结论，广泛应用于智能诊断、规则验证、自动定理证明等场景。本章将从基础的命题逻辑入手，带你掌握命题联结词运算、合式公式构造与重言式判定，学会用符号化语言表达简单逻辑关系；进而深入谓词逻辑，理解量词、谓词与个体变元的协同作用，突破命题逻辑的表达局限，能够处理更复杂的量化关系与多元关系。在此基础上，我们将聚焦归结推理这一自动推理的核心技术，拆解子句集化简、合一运算与归结规则的实现逻辑，掌握从逻辑公式到机器可执行推理的完整流程。通过命题逻辑约束求解、谓词逻辑关系建模、归结反演证明三大案例任务，你将亲手实现自动化推理系统，体会逻辑推理从抽象理论到工程实践的转化过程，为后续更复杂的智能决策系统开发奠定坚实的逻辑基础。

学习目标

- (1) 掌握命题逻辑核心知识，包括命题联结词运算、合式公式构造规则，能熟练判定重言式，并用符号化语言精准表达自然语言命题。
- (2) 理解谓词逻辑的关键要素，明确个体词、谓词、量词的作用，掌握谓词公式的等值变换与范式转换，能处理复杂量化关系与多元关系的形式化表达。
- (3) 深入掌握归结推理的核心流程，熟练运用子句集化简、合一运算与归结规则，理解从逻辑公式到机器可执行推理的完整转化逻辑。
- (4) 具备用 Python 实现自动化推理系统的能力，能独立完成命题逻辑约束求解、谓词逻辑关系建模及归结反演证明的代码开发与调试。
- (5) 培养从现实场景抽象逻辑规则的思维，能运用逻辑推理算法解决智能诊断、规则验证等复杂问题，掌握算法性能量化分析与测试评估方法。

3.1 理论知识介绍

3.1.1 命题逻辑

命题逻辑 (propositional logic)，亦称命题演算或陈述逻辑，是形式逻辑体系中的基础分支，专注于研究原子命题间的真值关系及其通过逻辑联结词 ($\neg$ 、 $\wedge$ 、 $\vee$ 、 $\rightarrow$ 、 $\leftrightarrow$) 构成的复合命题推理规则。其核心特征在于仅关注命题的二值性 (真/假)，完全剥离命题的语义内容，通过符号化方法构建纯粹的逻辑结构系统。

在逻辑推理领域，命题逻辑为形式化验证提供了基础框架：在数学证明中，它通过重言式 (如排中律) 为数学证明的严谨性提供公理化支撑；在计算机科学中，其不仅是数字电路门级设计、程序正确性验证的理论根基，更是构建知识表示系统 (如产生式规则库) 与自动推理引擎的核心形式化工具。例如，硬件描述语言本质上是对命题逻辑门级模型的符号化表达，而专家系统的规则引擎则依赖命题逻辑实现事实匹配与冲突消解。

命题逻辑的方法论价值在于将复杂推理过程进行模块化分解：通过原子命题的有限组合与真值演算规则，既可验证论证的有效性 (如归谬法证明)，又能构建可解释性推理模型。这种基于真值函数的分析范式，为人工智能系统实现可追踪、可验证的逻辑推理能力奠定了形式化基础。

1. 命题

命题是逻辑学的基本单元，其本质为具有排他性真值的陈述句，需满足双重判定标准：首先必须是陈述句式 (祈使句、疑问句与感叹句均不构成命题)，其次必须能够明确赋予真值 (真或假)，且两者互斥。具体而言，命题的真值由陈述内容与客观事实的一致性决定：与事实相符者为真命题 (真值标记为T)，相悖者为假命题 (真值标记为F)。在形式化表达中，真值通常以符号T (True) 与F (False) 表示，或映射为二进制值1与0，由此构成的逻辑体系称为二值逻辑系统。

有效命题和非命题示例如下：

(1) 有效命题：

“雪是白的”（事实性陈述，真值为T）。

“任一偶数可表示为两个素数之和”（哥德巴赫猜想，其真值尚未证实但仍属命题范畴）。

(2) 非命题：

“请关闭窗户”（祈使句，无真值属性）。

“ $1+101=110$ ”（数学命题，其真值依赖上下文：十进制中为假，二进制中为真）。

需特别指出，命题的真值唯一性是其核心特征：其真值必须且只能为真或假，不存在第三种状态。例如，尽管“明天会下雨”的真值在当下不可知，但其作为陈述句仍构成命题，因为未来某一时刻必然可判定其真伪。这种基于二值性的形式化抽象，使得命题逻辑能够剥离语义复杂性，专注于推理结构的有效性验证。

1) 命题变项

命题符号化是构建逻辑演算系统的形式化基础。在形式逻辑体系中，约定使用大写字母（如 P 、 Q ）作为命题变项（propositional variable），其本质是逻辑公式的抽象占位符。命题变项与具体命题的核心区别在于真值确定性：

命题：实例化的陈述句，具有确定的真值（如 P 表示“雪是白的”，其真值恒为T）。

命题变项：未赋值的逻辑符号，其真值处于未定状态（如 P 可表示任意命题），需通过具体命题代入实现语义绑定。

二者的关系可类比代数系统中的常量与变量：命题相当于确定值的常数（如数字5），其真值在形式系统中固定不变。命题变项则类似于代数变量 x ，需通过赋值操作（如 $x=3$ ）才能参与运算。

在逻辑演算过程中，命题与命题变项遵循统一的形式化处理原则：当命题变项被具体命题实例化后，其运算规则与原子命题完全一致。这种抽象层设计使得逻辑系统既能处理具体命题的语义验证，又能开展基于符号的通用推理演算。例如，在分析公式 $P \vee \neg P$ 时，无论 P 被赋值为“雪是白的”还是其他命题，该公式始终为重言式。这一特性凸显了命题逻辑的形式化本质——通过符号抽象剥离具体语义，专注于推理结构的有效性分析。

2) 原子命题和复合命题

命题逻辑将陈述句分为原子命题与复合命题两类。

(1) 原子命题。原子命题（亦称简单命题）是最小逻辑单元，其定义包含两个核心约束：

不可分割性：不包含任何逻辑联结词（如 $\neg$ 、 $\wedge$ 、 $\vee$ ），如“雪是白的”或“ $1+1=2$ ”均为原子命题，若进一步拆分则失去命题属性。

整体性处理：尽管原子命题可能包含主谓结构（如“张三学英语”），但在命题逻辑中不解析其内部成分，仅作为真值载体参与运算。

(2) 复合命题。复合命题（分子命题）由原子命题通过联结词组合而成，其真值由原子命题真值与联结词真值表共同决定。例如：

复合命题“雪是白的 $\wedge$ $1+1=2$ ”由两个原子命题通过合取词（ $\wedge$ ）联结，仅当两原子命题均为真时复合命题为真。

命题“张三学英语 $\vee$ 李四学日语”的真值则遵循析取词（ $\vee$ ）的语义规则。

在数理逻辑框架下，命题的本质被形式化为真值函数：逻辑系统不关注命题的具体语义（如“雪是白的”为何为真），而是研究其真值（T/F）的组合规律。例如，命题逻辑仅关心公式 $P \rightarrow Q$ 在 $P=T$ 、 $Q=F$ 时是否矛盾，而不解释 P 、 Q 的实际含义。这种形式化剥离使得逻辑演算能够独立于具体学科知识，专注于推理结构的有效性验证。

2. 命题联结词及真值表

逻辑联结词是构建复合命题的形式化工具，其重要性相当于代数系统中的算术运算符（如+、-、 $\times$ 、 $\div$ ）之于数值运算。通过联结词组合原子命题生成复合命题，极大扩展了命题逻辑的表达能力，其核心价值在于严格定义了复合命题的真值判定规则——复合命题的真值完全由原子命题的真值组合与联结词运算规则决定。需要强调的是，逻辑联结词与自然语言连接词存在本质差异：自然语言中的“和”“或”常隐含语境依赖（如时序关系或情感色彩），而逻辑联结词通过真值表明确定定义语义，消除歧义性。

命题逻辑系统定义五类基本联结词：

- （1）否定词（ $\neg$ ）：一元运算符，表示命题真值取反。
- （2）合取词（ $\wedge$ ）：二元运算符，表示逻辑“与”关系。
- （3）析取词（ $\vee$ ）：二元运算符，表示逻辑“或”关系。
- （4）蕴含词（ $\rightarrow$ ）：二元运算符，表示“如果...则...”。
- （5）双条件词（ $\leftrightarrow$ ）：二元运算符，表示逻辑等价关系。

每个联结词均通过真值表严格定义其运算规则。例如， $P \wedge Q$ 仅当 P 、 Q 均为真时结果为真，这与自然语言中“并且”的宽泛用法形成对比——逻辑合取不包含因果或时序关联，仅关注真值组合，这种形式化定义确保了推理过程的数学严谨性。

1) 否定词

否定词“ $\neg$ ”是个一元联结词，亦称否定符号。一个命题 P 加上否定词就形成了一个新的命题，记作 $\neg P$ ，这个新命题是命题的否定，读作非 P 。

否定词的真值规定如下：若命题 P 的真值为真，那么 $\neg P$ 的真值就为假；若 $\neg P$ 的真值为假，那么 $\neg P$ 的真值就为真。 P 与 $\neg P$ 间的真值关系，常常使用称作真值表的一种表格来表示，见表3-1。

也可将表3-1看作是对 P 的定义。它表明了 $\neg P$ 的真值如何依赖于 P 的真值。真值表描述了命题之间的真值关系，很直观，当命题变项的个数不多时，也很容易建立，真值表是命题逻辑里研究真值关系的重要工具。

表 3-1 否定词 $\neg$ 的真值表

P	$\neg P$
T	F
F	T

2) 合取词

合取词“ $\wedge$ ”是个二元命题联结词，亦称合取符号。将两个命题 P 、 Q 联结起来，构成一个新的命题 $P \wedge Q$ ，读作 P 、 Q 的合取，也可读作 P 与 Q 。这个新命题的真值与构成它的命题 P 、 Q 的真值间的关系，由合取词真值表来规定，见表3-2。

表3-2指出，只有当两个命题变项 $P=T$ 、 $Q=T$ 时方有 $P \wedge Q=T$ ，而 P 、 Q 只要有一为 F ，则 $P \wedge Q=F$ 。这样看来， $P \wedge Q$ 可用来表示日常用语 P 与 Q ，或 P 并且 Q 。

表 3-2 合取词 $\wedge$ 的真值表

P	Q	$P \wedge Q$
F	F	F
F	T	F
T	F	F
T	T	T

3) 析取词

析取词“ $\vee$ ”是个二元命题联结词，亦称析取符号。将两个命题 P 、 Q 联结起来，构成一个新的命题 $P \vee Q$ ，读作 P 、 Q 的析取，也读作 P 或 Q 。这个新命题的真值与构成它的命题 P 、 Q 的真值间的关系，由析取词真值表来规定，见表3-3。

表 3-3 析取词 $\vee$ 的真值表

P	Q	$P \vee Q$
F	F	F
F	T	T
T	F	T
T	T	T

表3-3指出，当 P 、 Q 有一取值为T时， $P \vee Q$ 便为T。仅当 P 、 Q 均取F值时， $P \vee Q$ 方为F。这就是析取词的定义， $P \vee Q$ 可用来表示自然用语 P 或 Q 。

4) 蕴含词

蕴涵词“ $\rightarrow$ ”也是个二元命题联结词，亦称推断符号。将两个命题 P 、 Q 联结起来，构成一个新的命题 $P \rightarrow Q$ ，读作如果 P 则 Q ，或读作 P 蕴涵 Q 、如果 P 那么 Q ，其中 P 称前件（前项、条件）， Q 称后件（后项、结论）。

规定只有当 P 为T而 Q 为F时， $P \rightarrow Q = F$ 。而 $P = F$ 、 Q 任意，或 $P = T$ 、 $Q = T$ 时， $P \rightarrow Q$ 均取值为T。真值表见表3-4。引入 $\rightarrow$ 的目的是希望用来描述命题间的推理，表示因果关系。实际上，表3-4说明了：

$P \rightarrow Q = T$ 下，若 $P = T$ 必有 $Q = T$ ，而不会出现 $Q = F$ ，这表明 $P \rightarrow Q$ 体现了 P 是 Q 成立的充分条件。

$P \rightarrow Q = T$ 下，若 $P = F$ 可有 $Q = T$ ，这表明 $P \rightarrow Q$ 体现了 P 不必是 Q 成立的必要条件。

表 3-4 蕴含词 $\rightarrow$ 的真值表

P	Q	$P \rightarrow Q$
F	F	T
F	T	T
T	F	F
T	T	T

使用 $P \rightarrow Q$ 能描述推理。即 $P \rightarrow Q$ 为真时，只要 P 为真必有 Q 真，而不能出现 P 真而 Q 假就够了。至于 P 为假时， Q 取真取假，并不违背 P 为真时 Q 必真。从而仍可规定 P 为假时， $P \rightarrow Q$ 取真。这当然只是对 $P \rightarrow Q$ 的一种说明，而从逻辑上说，本可按真值表定义 $P \rightarrow Q$ ，可不必涉及具体含义。另外，当 $P = F$ 时对 $P \rightarrow Q$ 真值的不同定义方式将给推理的讨论带来不同的表示形式，也是允许的。

$P \vee Q$ 的真值表（见表3-5）与表3-4中 $P \rightarrow Q$ 的真值表完全一致。这表明在所有真值组合下，蕴含式 $P \rightarrow Q$ 与析取式 $\neg P \vee Q$ 具有完全相同的逻辑真值。因此，我们可以建立等值关系 $P \rightarrow Q \equiv \neg P \vee Q$ （等号

表示逻辑等价)。这个重要的等价关系揭示了蕴含运算实际上可以通过否定和析取运算的组合来表达,同时也说明“如果 P 则 Q ”这一条件命题在逻辑上完全等价于“非 P 或 Q ”这一选择命题。

表 3-5 析取式 $\neg P \vee Q$ 的真值表

P	Q	$\neg P \vee Q$
F	F	T
F	T	T
T	F	F
T	T	T

逻辑蕴涵词“ $\rightarrow$ ”与自然语言中的“如果…那么…”结构存在部分对应关系,均可表达因果关联。但在形式逻辑中,即使命题 P 与 Q 毫无语义关联, $P \rightarrow Q$ 的表达式仍然具有明确的逻辑真值。特别当 P 为假时,无论 Q 的真值如何, $P \rightarrow Q$ 恒为真——这一特性与日常语言的使用习惯存在显著差异,体现了形式逻辑对命题关系的抽象处理方式。这种严格的形式化定义正是逻辑系统能够进行精确推理的基础。

5) 双条件词

双条件词“ $\leftrightarrow$ ”同样是个二元命题联结词,亦称等价符号。将两个命题 P 、 Q 联结起来构成新命题 $P \leftrightarrow Q$,读作 P 当且仅当 Q ,或读作 P 等价于 Q 。这个新命题的真值与 P 、 Q 真值间的关系,由双条件词的真值表来规定,见表3-6。

表 3-6 双条件词 $\leftrightarrow$ 的真值表

P	Q	$P \leftrightarrow Q$
F	F	T
F	T	F
T	F	F
T	T	T

表3-6指出,只有当两个命题 P 、 Q 的真值相同或 $P=Q$ 时, $P \leftrightarrow Q$ 的真值方为T。而当 P 、 Q 的真值不同时, $P \leftrightarrow Q=F$ 。

若建立 $(P \rightarrow Q) \wedge (Q \rightarrow P)$ 的真值表,就可发现 $(P \rightarrow Q) \wedge (Q \rightarrow P)$ 和 $P \leftrightarrow Q$ 有相同的真值,于是 $(P \rightarrow Q) \wedge (Q \rightarrow P) = P \leftrightarrow Q$ 。

3. 合式公式

合式公式(well-formed formula, WFF)是命题逻辑系统严格定义的形式化命题表达式,其合法性需通过形式语法规则严格判定。合式公式的递归定义如下:

- (1) 原子命题:所有简单命题(如 P 、 Q)均为合式公式。
- (2) 否定规则:若 A 是合式公式,则 $\neg A$ 亦是合式公式。
- (3) 联结规则:若 A 、 B 是合式公式,则 $(A \wedge B)$ 、 $(A \vee B)$ 、 $(A \rightarrow B)$ 、 (AB) 均为合式公式。
- (4) 闭包原则:仅有限次应用上述规则生成的符号串为合法合式公式。

该定义通过递归结构确保公式的合法性:基础情形(原子命题)与归纳步骤(联结操作)共同构成完备的形式语法体系。例如,公式 $(P \rightarrow (Q \vee R))$ 的合法性可通过逐层分解验证:

- (1) $Q \vee R$ 由原子命题 Q 、 R 经析取构成(规则3)。
- (2) $Q \vee R$ 作为合式公式参与蕴含运算(规则3)。

非合式公式示例：

(3) $P \vee Q \vee$ (运算符缺失操作数)。

(4) $(P \rightarrow (\wedge Q))$ (联结词 $\wedge$ 缺少左操作数)。

在实际书写中, 可通过运算符优先级规则减少括号冗余:

(1) 优先级降序: $\neg > \wedge > \vee > \rightarrow >$ 。

(2) 同级运算符采用左结合性, 例如: $P \rightarrow Q \vee R$ 等价于 $P \rightarrow (Q \vee R)$, $P \wedge Q \vee R$ 等价于 $(P \wedge Q) \vee R$ 。

此形式化体系确保每个公式具有唯一语法树结构, 为真值计算与逻辑等价变换提供可判定的形式基础。

4. 重言式

1) 定义

命题公式按其真值特性可分为三类严格定义的形式:

永真式(重言式): 若公式在命题变项的任意真值赋值下均为真, 则称为永真式(如 $P \vee P$)。

可满足式: 若存在至少一种真值赋值使公式为真, 则该公式可满足(如 $P \vee Q$ 在 $\{P=T, Q=F\}$ 时为真)。

矛盾式(永假式): 若公式在所有真值赋值下均为假, 则称为矛盾式(如 $P \wedge P$)。

三类公式的相互关系遵循形式化逻辑的基本法则:

(1) 公式 A 为永真式, 当且仅当其否定式 $\neg A$ 为矛盾式;

(2) 公式 A 可满足, 当且仅当 A 不是矛盾式;

(3) 不可满足的公式必为矛盾式;

(4) 非矛盾式必存在使其为真的赋值(即可满足)。

永真式在逻辑联结运算下具有封闭性: 由 $\vee$ 、 $\wedge$ 、 $\rightarrow$ 联结的永真式复合后仍保持永真性。这种特性为逻辑系统的完备性提供了形式化保障。需要强调的是, 永真式与可满足式构成包含关系——所有永真式均为可满足式, 但可满足式未必是永真式。

2) 代入规则

代入规则是保持逻辑公式永真性的关键操作: 若公式 A 是重言式, 则对其命题变项进行合法代入后得到的新公式 B 仍为重言式。此规则的成立需满足两个严格条件:

原子性约束: 被替换对象必须是原子命题变项(如 P 、 Q), 禁止对复合命题(如 $P \wedge Q$) 进行整体替换。例如, 可将公式中的原子命题 P 替换为复合公式 $R \wedge S$, 但不可将已有的 $R \wedge S$ 替换为原子命题 P 。

一致性约束: 对于公式中的任意命题变项, 其所有出现都必须被统一替换为相同的目标公式。例如, 将公式 A 中的变项 P 替换为 Q , 则 A 内所有 P 实例均需替换为 Q 。

代数系统与逻辑系统在代入规则上具有同构性: 正如代数等式 $(a+b)^2 = a^2 + 2ab + b^2$ 允许用 cd 替换 a 而保持等式成立, 但将 $a+b$ 整体替换为 cd 则会破坏等式结构。同理, 逻辑代入必须遵守原子替换原则, 否则将破坏公式的永真性。例如, 对重言式 $P \vee \neg P$ 进行合法代入(如用 Q 替换 P) 仍得重言式 $Q \vee \neg Q$; 但若仅替换部分 P 实例(如将 $P \vee \neg P$ 改为 $Q \vee \neg P$), 则破坏重言式结构。

5. 命题逻辑的等值和推理演算

推理形式与推理演算是数理逻辑体系的核心研究对象, 其形式化研究始于基本概念的严格定义。命题等值演算本质上是推理演算的特殊形式, 其有效性可通过蕴含词($\rightarrow$) 联结前提与结论构成的推理形式加以验证。演绎推理的核心特征在于结论必然性: 当推理形式满足前提真则结论必然真时, 该推理

形式方为有效。

重言式（如 $P \rightarrow P \vee Q$ ）在逻辑系统中具有双重价值：

- （1）作为基本逻辑规律，构成推理规则的公理基础；
- （2）作为有效推理的形式化表征，确保结论的必然导出。

命题等值式（如 $P \rightarrow Q \equiv \neg P \vee Q$ ）本质上是特殊类型的重言式，其揭示了不同逻辑表达式间的语义等价性。因此，重言式的研究与推理有效性的验证本质上是等价的学术进路——前者通过真值永恒性确立逻辑规律，后者通过形式推演保证结论可靠性。

1) 等值的定义

给定两个命题公式 A 和 B ，而 $P_1, \dots, P_n$ 是出现于 A 和 B 中的所有命题变项，那么公式 A 和 B 共有 2^n 个解释，若在其中的任一解释下，公式 A 和 B 的真值都相等，就称 A 和 B 是等值的（或称等价）。记作 $A=B$ 或 AB 。

显然，根据真值表就可以判明任何两个公式是否等值。

2) 等值定理

命题逻辑等值定理指出：两个公式 A 与 B 逻辑等值（记作 $A \equiv B$ ）的充要条件是双条件式 $A \leftrightarrow B$ 为永真式。该定理的数学表述为：对于包含原子命题 $P_1, \dots, P_n$ 的公式 A, B ，若在任意真值赋值下 A 与 B 的真值均相同，则 $A \leftrightarrow B$ 必为永真式；反之，若 $A \leftrightarrow B$ 为永真式，则 A 与 B 在所有解释下必然等值。

证明：

必要性：若 $A \equiv B$ ，则对所有真值赋值， A 与 B 真值相同。根据双条件词定义， $A \leftrightarrow B$ 在所有情况下为真，故为永真式。

充分性：若 $A \leftrightarrow B$ 为永真式，则其真值恒为T。依双条件词语义规则，仅当 A, B 真值相同时成立，故 $A \equiv B$ 。

3) 等值关系“ $\equiv$ ”满足以下基本性质

自反性：对任意公式 A ， $A \equiv A$ 。

对称性：若 $A \equiv B$ ，则 $B \equiv A$ 。

传递性：若 $A \equiv B$ 且 $B \equiv C$ ，则 $A \equiv C$ 。

需特别强调：“ $\equiv$ ”符号并非逻辑联结词，而是表示公式间语义等价关系的元语言符号。在合式公式的语法定义中，仅包含联结词（ $\neg, \wedge, \vee$ 等），而“ $\equiv$ ”属于语义层面的等价关系描述符。这一区分确保了逻辑系统语法与语义层面的严格分离。

3.1.2 谓词逻辑

谓词逻辑（predicate logic），亦称一阶逻辑（first-order logic），是形式逻辑体系的重要扩展。它在命题逻辑的基础上引入个体域、谓词与量词（ $\forall, \exists$ ）的概念，突破命题逻辑的原子命题限制，能够精确刻画个体对象的属性及其相互关系。与命题逻辑仅关注真值的二值性不同，谓词逻辑通过量化机制（如“对所有 x ”或“存在某个 x ”）实现对象层面的精细化描述。

在数学基础研究中，谓词逻辑为集合论的公理化、数论命题的形式化证明以及模型论的结构分析提供了严格的语言框架。其量化表达能力使得数学概念的抽象层次得以系统性提升，例如实数连续性的 ε - δ 定义本质上是嵌套量词结构的谓词公式。

计算机科学中，谓词逻辑构成关系数据库的理论内核——SQL查询语言可视为受限谓词逻辑的可执

行子集，而逻辑编程范式（如 Prolog）直接基于霍恩子句的谓词推理机制。在知识表示领域，谓词逻辑通过框架网络与产生式规则，支持复杂领域知识的结构化建模，为专家系统与自动推理引擎提供形式化基础。

谓词逻辑的方法论突破在于将推理过程解耦为三个维度：个体对象的取值范围（论域）、对象间关系的谓词描述，以及量词约束下的命题组合。这种分层建模思想不仅增强了逻辑系统的表达能力，更为可判定性证明与模型检测算法奠定了理论基础。

1. 谓词和个体词

1) 谓词

例：张三是学生，李四是学生。在命题逻辑里，这是两个不同的命题，只能分别以两个不同的符号如 p ， q 表示。然而这两个命题的共同点是，它们都有主词和谓词。主词“张三”“李四”是不同的，而谓词“是学生”是相同的。

现在强调它们的共同点。若以大写符号 P 表示“是学生”，这样两个命题的共同性可由 P 来体现了，但主词还需区别开来，便可把这两个命题分别写成 $P(\text{张三})$ 和 $P(\text{李四})$ 。明显地描述了这两个命题的共同点和不同点，一般地可引入变量 x 来表示主词，于是符号 $P(x)$ 就表示“ x 是学生”。通常把 $P(x)$ 称作谓词。

（1）谓词描述性定义：

一元谓词：在一个命题里，如果主词只有一个，这时表示该主词性质或属性的词便称作谓词。这是一元（目）谓词，以 $P(x)$ ， $Q(x)$ ， $\cdots$ 表示。

多元谓词：在一个命题里，如果主词多于一个，那么表示这几个主词间的关系的词称作谓词。这是多元谓词，以 $P(x, y)$ ， $Q(x, y)$ ， $P(x, y, z)$ ， $\cdots$ 表示。

（2）举例：

“张三和李四是表兄弟”，其中“是表兄弟”是谓词；“5大于3”，其中“大于”是谓词；“张三比李四高”，其中“比 $\cdots$ 高”是谓词；“天津位于北京的东南”，其中“位于 $\cdots$ 东南”是谓词；“ A 在 B 上”，其中“在 $\cdots$ 上”是谓词。

2) 个体词

个体词（主词）：个体词是一个命题里表示思维对象的词。 $P(\text{张三})$ 中的张三是个体词或称个体常项。谓词 $P(x)$ 中的变量 x 为个体变项或个体变元。

n 项（目、元）谓词：有 n 个个体的谓词 $P(x_1, \cdots, x_n)$ 称 n 项（目、元）谓词；如果 P 是已赋有确定含义的谓词，就称为谓词常项；如果 P 表示任一谓词时，就称为谓词变项。

个体域：将个体变元的变化范围称为个体域或论域，以 D 表示。论域是重要的概念，同一谓词在不同论域下的描述形式可能不同，所取的真值也可能不同。

约定：谓词逻辑的个体域除明确指明外，都认为是包括一切事物的一个最广的集合。谓词变项的变化范围，不做特别声明时，指一切关系或一切性质的集合。

3) 谓词的定義

谓词在逻辑系统中具有明确的数学定义：它既可以表示单个个体的属性，也可以描述多个个体之间的关系。从形式化角度看，谓词实际上是从个体域到真值集合 $\{T, F\}$ 的一个映射函数。

读者可以通过具体例子来理解这个概念：比如 $P(x)$ 表示“ x 是有理数”，当 $x=3$ 时， $P(3)$ 就是一个取值为 T 的具体命题；而 $Q(x, y)$ 表示“ x 大于 y ”，则 $Q(2, 3)$ 就是一个取值为 F 的命题。需要注意的

是,谓词本身(如 P 、 Q)在没有具体实例化时只是命题形式,其真值取决于个体变元的取值和论域范围。例如“房子是黄色的”既可以表示为一元谓词YELLOW (HOUSE),也可以通过二元谓词VALUE (COLOR, HOUSE)来表达,后者更加抽象地体现了谓词作为真值映射的本质特征。

4) 谓词逻辑与命题逻辑

谓词逻辑与命题逻辑之间存在着深刻的联系与区别。读者应当理解,谓词逻辑可以视为命题逻辑的扩展,而命题逻辑则是谓词逻辑在零元谓词情况下的特例。在谓词逻辑中,命题逻辑的基本概念和规则大多仍然适用,包括逻辑联结词的使用和部分等价式的推导。然而,谓词逻辑引入了量词、谓词和个体变元等新要素,特别是当涉及无限论域时,其复杂性显著增加。一个典型的差异体现在判定问题上:命题逻辑中的重言式可以通过真值表法有效判定,而谓词逻辑中却不存在通用的算法来判断任意公式的普遍有效性。这种计算复杂性的跃升,正是谓词逻辑表达能力增强所付出的代价。

2. 谓词公式

1) 谓词公式的解释

在谓词逻辑中, n 元谓词 P 被定义为从论域 D 的 n 次笛卡儿积到真值集合 $\{T, F\}$ 的映射,记作 $P(x_1, x_2, \dots, x_n)$,其中 x_i 可以是个体常量、变元或函数。读者需要注意,谓词公式的真值取决于以下两个关键因素:

- 对个体常量和函数的赋值(值指派),即确定它们在论域 D 中的具体取值;
- 对谓词本身的真值指派,即确定谓词在不同输入下的真值输出。

谓词公式的解释 I 是指在非空论域 D 上,对公式中所有符号进行系统赋值的过程:

- 为每个个体常量指定 D 中的一个具体元素;
- 为每个 n 元函数指定 $D^n \rightarrow D$ 的映射;
- 为每个 n 元谓词指定 $D^n \rightarrow \{T, F\}$ 的映射。

特别需要强调的是,同一个谓词公式在不同解释下可能具有不同的真值。这种特性引出了谓词公式的三个重要分类:永真式(在所有解释下为真)、永假式(在所有解释下为假)、可满足式(至少存在一个解释使其为真)。

读者应当理解,这种解释机制是谓词逻辑比命题逻辑更具表达力的关键所在,同时也带来了判定问题的计算复杂性。

2) 谓词公式的永真性和可满足性

谓词公式的永真性:如果谓词公式 P 对非空个体域 D 上的任一解释都取得真值 T ,则称 P 在 D 上是永真的;如果 P 在任何非空个体域上均是永真的,则称 P 永真。

谓词公式的可满足性:对于谓词公式 P ,如果至少存在 D 上的一个解释,使公式 P 在此解释下的真值为 T ,则称公式 P 在 D 上是可满足的。谓词公式的可满足性也称为相容性。

谓词公式的永假性:如果谓词公式 P 对非空个体域 D 上的任一解释都取真值 F ,则称 P 在 D 上是永假的;如果 P 在任何非空个体域上均是永假的,则称 P 永假。谓词公式的永假性又称不可满足性或不相容。

3) 谓词公式的等价性和永真蕴含性

(1) 谓词公式的等价式可定义如下:

定义:设 P 与 Q 是 D 上的两个谓词公式,若对 D 上的任意解释, P 与 Q 都有相同的真值,则称 P 与 Q 在 D 上是等价的。如果 D 是任意非空个体域,则称 P 与 Q 是等价的,记作 $P \Leftrightarrow Q$ 。

- ① 双重否定律: $\neg \neg P \Leftrightarrow P$ 。
- ② 交换律: $(P \vee Q) \Leftrightarrow (Q \vee P)$, $(P \wedge Q) \Leftrightarrow (Q \wedge P)$ 。
- ③ 结合律: $(P \vee Q) \vee R \Leftrightarrow P \vee (Q \vee R)$, $(P \wedge Q) \wedge R \Leftrightarrow P \wedge (Q \wedge R)$ 。
- ④ 分配律: $P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$, $P \wedge (Q \vee R) \Leftrightarrow (P \wedge Q) \vee (P \wedge R)$ 。
- ⑤ 摩根定律: $\neg (P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$, $\neg (P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$ 。
- ⑥ 吸收律: $P \vee (P \wedge Q) \Leftrightarrow P$, $P \wedge (P \vee Q) \Leftrightarrow P$ 。
- ⑦ 补余律: $P \vee \neg P \Leftrightarrow T$, $P \wedge \neg P \Leftrightarrow F$ 。
- ⑧ 连词化归率: $P \rightarrow Q \Leftrightarrow \neg P \vee Q$, $P \leftrightarrow Q \Leftrightarrow (P \rightarrow Q) \wedge (Q \rightarrow P)$, $P \leftrightarrow Q \Leftrightarrow (P \wedge Q) \vee (\neg P \wedge \neg Q)$ 。
- ⑨ 量词转换率: $\neg (\exists x)P \Leftrightarrow (\forall x)(\neg P)$, $\neg (\forall x)P \Leftrightarrow (\exists x)(\neg P)$ 。
- ⑩ 量词分配率: $(\forall x)(P \wedge Q) \Leftrightarrow (\forall x)P \wedge (\forall x)Q$, $(\exists x)(P \vee Q) \Leftrightarrow (\exists x)P \vee (\exists x)Q$ 。

(2) 谓词公式的永真蕴涵式可定义如下:

定义: 对谓词公式 P 和 Q , 如果 $P \rightarrow Q$ 永真, 则称 P 永真蕴含 Q , 且称 Q 为 P 的逻辑结论, P 为 Q 的前提, 记作 $P \Rightarrow Q$ 。

常用的永真蕴含式如下:

- ① 化简式 $P \wedge Q \Rightarrow P$, $P \wedge Q \Rightarrow Q$ 。
- ② 附加式 $P \Rightarrow P \vee Q$, $Q \Rightarrow P \vee Q$ 。
- ③ 析取三段论 $\neg P$, $P \vee Q \Rightarrow Q$ 。
- ④ 假言推理 P , $P \rightarrow Q \Rightarrow Q$ 。
- ⑤ 拒取式 $\neg Q$, $P \rightarrow Q \Rightarrow \neg P$ 。
- ⑥ 假言三段论 $P \rightarrow Q$, $Q \rightarrow R \Rightarrow P \rightarrow R$ 。
- ⑦ 二难推理 $P \vee Q$, $P \rightarrow R$, $Q \rightarrow R \Rightarrow R$ 。

⑧ 全称固化 $(\forall x)P(x) \Rightarrow P(y)$, 其中, y 是个体域中的任一个体, 依此可消去谓词公式中的全称量词。

⑨ 存在固化 $(\exists x)P(x) \Rightarrow P(y)$, 其中, y 是个体域中某一个可以使 $P(y)$ 为真的个体, 依此可消去谓词公式中的存在量词。

4) 谓词公式的范式

范式是谓词公式的标准形式。在谓词逻辑中, 范式包括前束范式和Skolem范式两种。

(1) 前束范式: 所有量词均非否定地出现在公式的最前面; 所有量词的辖域为整个公式。如 $(\forall x)(\forall y)(\exists z)(P(x) \wedge Q(y, z) \vee R(x, z))$ 。

(2) Skolem范式: 前束范式, 所有的存在量词都在全称量词之前。如 $(\exists x)(\exists z)(\forall y)(P(x) \vee Q(y, z) \wedge R(x, z))$ 。

5) 置换与合一

在基于谓词的逻辑推理中, 经常会遇到谓词名称相同但个体参数不同的情况。此时需要通过置换 (substitution) 操作来实现匹配, 这个过程称为合一 (unification)。

(1) 置换是指用项 (常量、函数或变量) 替换变量的操作, 形式上表示为 $\{t_1/t_1, t_2/t_2, \dots, t_n/t_n\}$, 其中 x_i 是互不相同的变量, t_i 是替换 x_i 的项。置换必须遵循三个基本原则:

- 只能替换变量, 不能替换常量或函数;
- 被替换变量 x_i 不能出现在替换项 t_i 中 (避免循环);

- 不能存在相互置换 (如 $y/x, x/y$)。

(2) 最一般合一 (most general unifier, MGU) 是指能够通过复合置换得到所有其他合一的置换 σ , 即对任意合一 θ , 存在 λ 使得 $\theta = \sigma \circ \lambda$ 。MGU 的重要性在于它产生的实例最一般化, 有利于后续推理。

Robinson 合一算法是求解 MGU 的标准方法:

- 将谓词表示为表结构, 如 $P(x, f(y), B)$ 表示为 $(Px(fy) B)$;
- 检查两个谓词表长度和谓词名是否相同;
- 通过寻找并消除差别集 (difference set) 逐步构建置换。

该算法不仅能判断可合一性, 还能在可合一时给出 MGU。读者需要注意, 合一过程可能涉及变量重命名以避免冲突。

3. 函数和量词

1) 函数

在谓词逻辑中出现变量, 自然也会考虑引入函数。函数是某个体域 (不必是实数) 到另一个个体域的映射。不同于谓词: 将个体映射为真假值, 函数并不单独使用, 是嵌入在谓词中。

设 $\text{father}(x)$ 表示 x 的父亲, $P(x)$ 表示 “ x 是教师”, 那么 $P(\text{father}(x))$ 就表示 “ x 的父亲是教师” 这一命题。当 x 取特定值时, 该命题的真值即可确定。再如, “张三的父亲和李四的哥哥是同事” 这一陈述可以形式化为 $\text{COLLEAGUE}(\text{father}(\text{张三}), \text{brother}(\text{李四}))$, 其中 COLLEAGUE 是二元谓词, father 和 brother 都是个体函数。需要注意的是, 在谓词逻辑中, 函数符号通常采用小写字母表示, 如 f 、 g 或具有明确语义的 father 、 brother 等。这种表示方法既保持了形式化的严谨性, 又增强了表达的自然性。

2) 量词

量词是谓词逻辑中用于约束个体变元的重要运算符, 主要分为全称量词和存在量词两种基本类型。在某种意义上说, 这是一对相对立的词。

全称量词表示 “对于所有” 的概念, 符号化为 $\forall x$, 对应自然语言中的 “所有” “每一个” 等表述。例如 “所有事物都是运动的” 可形式化为 $\forall x P(x)$, 其中 $P(x)$ 表示 “ x 是运动的”。读者应当理解, $\forall x P(x)$ 为真当且仅当论域中每个 x 都使 $P(x)$ 为真。

存在量词表示 “存在至少一个” 的概念, 符号化为 $\exists x$, 对应自然语言中的 “存在” “有些” 等表述。例如 “有事物是动物” 可形式化为 $\exists x Q(x)$, 其中 $Q(x)$ 表示 “ x 是动物”。读者需要注意, $\exists x Q(x)$ 为真当且仅当论域中至少存在一个 x 使 $Q(x)$ 为真。这两种量词在逻辑上形成互补关系, 共同构成了谓词逻辑量化表达的基础框架。

3) 约束变元和自由变元

在一个含有量词的命题形式里, 区分个体词受量词的约束还是不受量词的约束是重要的。无论在定义合式公式以及对个体变元作代入时都需区分这两种情形。

若 $P(x)$ 表示 x 是有理数, 这时的变元 x 不受任何量词约束, 便称是自由的。而 $(\forall x) P(x)$ 中的两处出现的变元 x 都受量词 $\forall$ 的约束, 便称作约束变元, 受约束的变元也称被量词量化的变元。

命题形式 $(\forall x) P(x) \vee Q(y)$ 中, 变元 x 是约束的, 而变元 y 是自由的。

量词所约束的范围称为量词的辖域。如 $(\forall x) R(x, y)$ 中, $R(x, y)$ 是 $(\forall x)$ 的辖域; $(\exists x)((\forall y) P(x, y))$ 中, $P(x, y)$ 是 $(\forall y)$ 的辖域, $(\forall y) P(x, y)$ 是 $(\exists x)$ 的辖域。

3.1.3 归结推理

归结推理，也称为归结原则或归结演绎，是一种自动定理证明和逻辑推理的方法，它基于逻辑公式的归结规则来推导结论。归结推理不仅关注命题的真值，还通过将逻辑公式转化为一种标准形式（子句）并应用特定的推理规则，来探索命题之间的逻辑关系。

归结推理在逻辑推理中的应用非常广泛，它提供了一种系统化和机械化的方法来形式化和验证推理过程。在计算机科学领域，归结推理是许多自动定理证明器和逻辑编程语言（如Prolog）的核心，它允许计算机自动地推导和验证逻辑公式的真假。

在人工智能中，归结推理用于构建专家系统和知识表示系统，它使得机器能够模拟人类的推理过程，解决复杂的问题。在法律逻辑中，归结推理有助于自动化法律论证和案例分析，提高法律决策的效率和准确性。

总体而言，归结推理通过将逻辑公式转化为子句并应用归结规则，为确保推理过程的准确性和有效性提供了一种高效和自动化的方法。它允许我们将复杂的推理过程简化为一系列标准化的操作，从而帮助我们分析和构建更为复杂和精确的论证。归结推理的自动化特性使其成为现代逻辑推理和计算逻辑中不可或缺的一部分。

1. 子句集的化简

在谓词逻辑中，任何一个谓词公式都可以通过应用等价关系及推理规则化成相应的子句集。其化简步骤如下：

- (1) 消条件；
- (2) 减否定；
- (3) 量词变元一致；
- (4) 化前束；
- (5) 消存在；
- (6) 化skolem合取式；
- (7) 消全称；
- (8) 化子句；
- (9) 换变元。

下面是每个步骤的详细过程：

1) 消条件

目标：消去条件连接词“ $\rightarrow$ ”和“ $\leftrightarrow$ ”。

方法：反复使用等价公式： $P \rightarrow Q \Leftrightarrow \neg P \vee Q$, $P \leftrightarrow Q \Leftrightarrow (P \wedge Q) \vee (\neg P \wedge \neg Q)$ 。

2) 减否定

目标：将每个否定符号“ $\neg$ ”移到紧靠谓词的位置，使得每个否定符号最多只作用于一个谓词上。

方法：

双重否定律： $\neg \neg P \Leftrightarrow P$ 。

摩根定律： $\neg (P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$, $\neg (P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$ 。

量词转换率： $\neg (\exists x) P \Leftrightarrow (\forall x) (\neg P)$, $\neg (\forall x) P \Leftrightarrow (\exists x) (\neg P)$ 。

如将： $(\forall x) (\neg (\forall y) P(x, y) \vee \neg (\forall y) (\neg Q(x, y) \vee R(x, y)))$ 。

化为： $(\forall x) ((\exists y) \neg P(x, y) \vee (\exists y) (Q(x, y) \wedge \neg R(x, y)))$ 。

3) 量词变元一致

目标：使不同量词约束的变元有不同的名字。

方法：在一个量词的辖域内，把谓词公式中受该量词约束的变元全部用另外一个没有出现过的任意变元代替。

如将： $(\forall x)((\exists y)\neg P(x, y) \vee (\exists y)(Q(x, y) \wedge \neg R(x, y)))$

化为： $(\forall x)((\exists y)\neg P(x, y) \vee (\exists z)(Q(x, z) \wedge \neg R(x, z)))$ 。

4) 化前束

目标：把所有量词都移到公式的左边

方法：在移动时不能改变其相对顺序；由于第(3)步已对变元进行了标准化，每个量词都有自己的变元，这就消除了任何由变元引起冲突的可能，因此这种移动是可行的。

如将： $(\forall x)((\exists y)\neg P(x, y) \vee (\exists z)(Q(x, z) \wedge \neg R(x, z)))$

化为： $(\forall x)(\exists y)(\exists z)(\neg P(x, y) \vee (Q(x, z) \wedge \neg R(x, z)))$

5) 消存在

方法：消去存在量词时，需要区分以下两种情况：

(1) 若存在量词不出现于全称量词的辖域内（即它的左边没有全称量词），只要用一个新的个体常量替换受该存在量词约束的变元，就可消去该存在量词。

(2) 若存在量词位于一个或多个全称量词的辖域内，例如 $(\forall x_1) \cdots (\forall x_n)(\exists y)P(x_1, x_2, \dots, x_n, y)$ ，则需要用 skolem 函数 $f(x_1, x_2, \dots, x_n)$ 替换受该存在量词约束的变元 y ，然后再消去该存在量词。

如，上步所得公式： $(\forall x)(\exists y)(\exists z)(\neg P(x, y) \vee (Q(x, z) \wedge \neg R(x, z)))$ 中，存在量词 $(\exists y)$ 和 $(\exists z)$ 都位于 $(\forall x)$ 的辖域内，因此都需要用 skolem 函数来替换。

设替换 y 和 z 的 skolem 函数分别是 $f(x)$ 和 $g(x)$ ，则替换后的式子为 $(\forall x)(\neg P(x, f(x)) \vee (Q(x, g(x)) \wedge \neg R(x, g(x))))$ 。

注意：这种替换不影响原谓词公式的永假性。

6) 化skolem合取式

目标：将谓词公式化为如下 skolem 标准形式：

$$(\forall x_1) \cdots (\forall x_n) M(x_1, \dots, x_n)$$

$M(x_1, \dots, x_n)$ 是 skolem 标准型的母式，它由子句的合取所构成。

方法：使用等价关系 $P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$

如将： $(\forall x)(\neg P(x, f(x)) \vee (Q(x, g(x)) \wedge \neg R(x, g(x))))$

化为： $(\forall x)((\neg P(x, f(x)) \vee Q(x, g(x))) \wedge (\neg P(x, f(x)) \vee \neg R(x, g(x))))$ 。

7) 消全称

方法：由于母式中的全部变元均受全称量词的约束，并且全称量词的次序已无关紧要，因此可以省掉全称量词。但剩下的母式，仍假设其变元是被全称量词量化的。

如将： $(\forall x)((\neg P(x, f(x)) \vee Q(x, g(x))) \wedge (\neg P(x, f(x)) \vee \neg R(x, g(x))))$

化为： $(\neg P(x, f(x)) \vee Q(x, g(x))) \wedge (\neg P(x, f(x)) \vee \neg R(x, g(x)))$ 。

8) 化子句

目标：化为谓词之间相互析取的子句。

方法：在母式中消去所有合取词，把母式用子句集的形式表示出来。

如将： $(\neg P(x, f(x)) \vee Q(x, g(x))) \wedge (\neg P(x, f(x)) \vee \neg R(x, g(x)))$

化为：

子句 (1)： $\neg P(x, f(x)) \vee \neg R(x, g(x))$ 。

子句 (2)： $\neg P(x, f(x)) \vee Q(x, g(x))$ 。

9) 换变元

目标：对子句集中的某些变量重新命名，使任意两个子句中不出现相同的变量名。

如将： $\neg P(x, f(x)) \vee Q(x, g(x))$ 与 $\neg P(x, f(x)) \vee \neg R(x, g(x))$

化为： $\neg P(x, f(x)) \vee Q(x, g(x))$ $\neg P(y, f(y)) \vee \neg R(y, g(y))$ 。

2. 归结原理

1) 基本思想

归结原理是一种基本的逻辑推理方法，用于判断一个命题集合是否可满足，或者用于证明一个逻辑命题是否成立。它是一种基于逻辑推理规则的自动推理技术，广泛应用于人工智能、自动定理证明、程序验证等领域。以下是归结原理的详细解释：

归结原理的基本思想是通过逻辑推理规则将待证明命题（或者待解的问题）转化为一个简单的矛盾命题，从而推导出其否定式的空集，进而证明原命题成立。其关键在于不断应用归结规则，将待证明命题逐步简化为矛盾式。

2) 命题逻辑的归结

(1) 归结式的定义及性质。

定义：在命题逻辑中，一个归结式是两个子句的归结，其中一个子句包含了某个命题的肯定形式，而另一个子句包含了该命题的否定形式。通过合一操作，这两个子句可以归结为一个新的子句。

性质：

保持真值：如果两个子句都是真的，那么它们的归结也是真的。

无后效性：归结的结果只与归结的两个子句有关，与其他子句无关。

终止性：归结推理过程会在有限步骤内终止，因为每次归结操作都会生成一个新的子句，而子句的长度是有限的。

(2) 归结反演的基本思想。

归结反演是一种推理方法，它通过将待证明的目标命题转化为归结式的形式，然后反向推导出原始的命题，从而证明原命题成立。其基本思想如下：

目标命题的转化：将待证明的目标命题转化为归结式的形式，即将目标命题的否定形式作为归结式的一个子句，然后在子句集中寻找与之归结的子句。

反向推导：一旦找到与目标归结的子句，就可以通过合一操作和归结规则，反向推导出原命题的肯定形式。如果成功得到空子句，则说明原命题成立。

搜索策略：在归结反演的过程中，可以采用启发式搜索策略，例如选择合适的子句进行归结，或者采用剪枝策略来减少搜索空间，以提高搜索效率。

优化策略：为了提高归结反演的效率，可以采用一些优化策略，例如单元子句消解，优先应用包含单个命题的子句进行归结，以减少搜索空间；子句选择策略，选择合适的子句进行归结，可以采用启发式算法或者基于知识的方法来选择最有可能导致矛盾的子句。

3.2 案例描述

3.2.1 案例背景介绍

人工智能技术的快速发展使逻辑推理算法成为该领域的重要研究方向。作为模拟人类思维过程的基础工具，逻辑推理算法能够根据已知条件进行有效推断，在自然语言处理、计算机视觉和机器人等关键技术中发挥着核心作用。读者在实际应用中常会遇到复杂的推理决策问题，如案件侦破、医疗诊断和智能推荐等场景，这些都需要对多源信息进行逻辑分析和综合判断。通过系统学习命题逻辑、谓词逻辑和归结推理等基本方法，读者可以掌握处理复杂逻辑问题的有效工具，为构建智能系统奠定理论基础。

3.2.2 案例技术分析

1. 命题逻辑基础

学习原子命题与复合命题的构成，掌握逻辑联结词（ $\neg$ 、 $\wedge$ 、 $\vee$ 、 $\rightarrow$ 、 $\leftrightarrow$ ）的真值表运算；理解推理有效性判定方法，包括真值表技术、公式转换法和演绎推理规则（如假言推理、选言三段论）；通过Python实现命题逻辑的自动化验证，例如构建真值表生成器或推理规则验证程序。

2. 谓词逻辑扩展

掌握量词（ $\forall$ 、 $\exists$ ）的使用及约束变元处理，能够将自然语言命题形式化为谓词表达式；区分一元谓词与多元谓词，理解函数项在知识表示中的作用；通过Python实现简单的谓词实例化与匹配，例如构建知识图谱的基础推理模块。

3. 归结推理方法

学习子句集转换方法（skolem标准化、消解全称量词等）；掌握归结反演的核心算法，能够通过消解互补文字推导空子句；使用Python实现Robinson归结算法，处理简单的定理证明问题。

3.3 设计与实现

3.3.1 命题逻辑案例

1. 案例任务

三个老师问题：已知A、B、C，三人是小学老师，各教两门课，互不重复，共有如下六门课，语文（Chinese）、数学（Math）、政治（Politics）、地理（Geography）、音乐（Music）和美术（Art），已经知道：

- 政治老师和数学老师是邻居。
- 地理老师比语文老师年龄大。
- B最年轻。
- A经常给地理老师和数学老师讲他看过的文学作品。
- B经常和音乐老师、语文老师一起游泳。

要求：编程判断A、B、C各教哪门课。

2. 环境

开发工具为Python 3.12，推荐使用PyCharm或Visual Studio Code集成开发环境以提高编码效率。

相关库：itertools（用permutations（）函数来生成六门课的所有排列）。

3. 步骤

（1）从itertools库导入permutations。

```
from itertools import permutations
```

（2）定义老师和课程。

```
teachers = ['A', 'B', 'C']  
subjects = ['Chinese', 'Math', 'Politics', 'Geography', 'Music', 'Art']
```

（3）定义并实现条件检查函数。

```
def check_conditions(assignment):  
    # 政治老师和数学老师是邻居，说明政治和数学老师不可能是同一个老师  
    if 'Politics' in assignment['A'] and 'Math' in assignment['A']:  
        return False  
    if 'Politics' in assignment['B'] and 'Math' in assignment['B']:  
        return False  
    if 'Politics' in assignment['C'] and 'Math' in assignment['C']:  
        return False  
  
    # 地理老师比语文老师年龄大，说明地理和语文不可能是同一个老师教  
    if 'Geography' in assignment['A'] and 'Chinese' in assignment['A']:  
        return False  
    if 'Geography' in assignment['B'] and 'Chinese' in assignment['B']:  
        return False  
    if 'Geography' in assignment['C'] and 'Chinese' in assignment['C']:  
        return False  
  
    # B最年轻，且地理老师比语文老师年龄大，说明B不是地理老师  
    if 'Geography' in assignment['B']:  
        return False  
  
    # A经常给地理老师和数学老师讲他看过的文学作品，说明A不可能是地理老师和数学老师  
    if 'Geography' in assignment['A'] or 'Math' in assignment['A']:  
        return False  
  
    # B经常和音乐老师、语文老师一起游泳，说明B不可能是音乐老师和语文老师  
    if 'Music' in assignment['B'] or 'Chinese' in assignment['B']:  
        return False  
  
    return True
```

(4) 主函数。

```
# 尝试所有可能的课程分配
def main():
    for perm in permutations(subjects):
        assignments = {'A': perm[0:2], 'B': perm[2:4], 'C': perm[4:6]}
        if check_conditions(assignments):
            print("A teaches:", assignments['A'])
            print("B teaches:", assignments['B'])
            print("C teaches:", assignments['C'])
            break

if __name__ == '__main__':
    main()
```

4. 代码运行结果

```
A teaches: ('Chinese', 'Politics')
B teaches: ('Math', 'Art')
C teaches: ('Geography', 'Music')
```

3.3.2 谓词逻辑案例

1. 案例任务

假设有一个图书馆系统，其中包含书籍（Books）、作者（Authors）、读者（Readers）和借（Borrowing）的概念。我们定义以下谓词：

- Authored (x, y): 表示个体 x 写了书籍 y。
- Reads (z, w): 表示读者 z 阅读了书籍 w。
- Borrowed (a, b): 表示读者 a 从图书馆借了书籍 b。

现在，我们有以下事实：

- A写了书籍BookA。
- B写了书籍BookB。
- A阅读了书籍BookA。
- B阅读了书籍BookB。
- C阅读了书籍BookA。
- B从图书馆借了书籍BookB。
- C从图书馆借了书籍BookA。

请回答以下问题：

问题1：所有的作者都读过自己写的书？

问题2：存在非作者读这本书吗？

问题3：如果一个人借了一本书，那么这个人就是这本书的作者吗？

请使用谓词逻辑表达式来表达上述问题，并尝试解答它们。

2. 环境

开发工具为Python 3.12，推荐使用PyCharm或Visual Studio Code集成开发环境以提高编码效率。

3. 步骤

(1) 定义事实。

我们可将事实转换为用谓词逻辑表示：

Authored (A, BookA)

Authored (B, BookB)

Reads (A, BookA)

Reads (B, BookB)

Reads (C, BookA)

Borrowed (B, BookB)

Borrowed (C, BookA)

```
facts = {
    "Authored": [("A", "BookA"), ("B", "BookB")],
    "Reads": [("A", "BookA"), ("B", "BookB"), ("C", "BookA")],
    "Borrowed": [("C", "BookA"), ("B", "BookB")]
}
```

(2) 问题1：所有的作者都读过自己写的书？

谓词逻辑表达式： $\forall x \forall y (\text{Authored}(x, y) \rightarrow \text{Reads}(x, y))$ 。

```
def all_authors_read_own_books(facts):
    authored = facts["Authored"]
    reads = facts["Reads"]
    # 检查每个作者是否读过他们自己写的书
    for author, book in authored:
        if (author, book) not in reads:
            return False
    return True
```

(3) 问题2：存在非作者读这本书吗？

谓词逻辑表达式： $\exists x \exists y (\text{Reads}(x, y) \wedge \neg \text{Authored}(x, y))$ 。

```
def exists_non_author_reads_book(facts):
    authors = {author for author, _ in facts["Authored"]} # A B
    reads = facts["Reads"] # [("A", "BookA"), ("B", "BookB"), ("C", "BookA")]
    # 由于所有作者都会读自己写的书，所以只要有读者不在作者列表内就能表明存在非作者读这本书
    # 检查是否有读者读了书，但他们不是作者
    for reader, _ in reads:
        if reader not in authors:
            return True
    return False
```

(4) 问题3: 如果一个人借了一本书, 那么这个人就是这本书的作者吗?

谓词逻辑表达式: $\forall x \forall y (Borrowed(x, y) \rightarrow Authored(x, y))$ 。

```
def borrowed_implies_authored(facts):
    borrowers = facts["Borrowed"]
    authors = facts["Authored"]

    for x in borrowers:
        if x not in authors:
            return False
    return True
```

(5) 主函数。

```
def main():
    # 运行代码并输出结果
    print(" 问题 1: 所有的作者都读过自己写的书? ", all_authors_read_own_books(facts))
    print(" 问题 2: 存在非作者读这本书吗? ", exists_non_author_reads_book(facts))
    print(" 问题 3: 如果一个人借阅了一本书, 那么这个人就是这本书的作者吗? ",
          borrowed_implies_authored(facts))

if __name__ == '__main__':
    main()
```

4. 代码运行结果

问题 1: 所有的作者都读过自己写的书? True

问题 2: 存在非作者读这本书吗? True

问题 3: 如果一个人借阅了一本书, 那么这个人就是这本书的作者吗? False

3.3.3 归结推理案例

1. 案例任务

小华是一名大学生, 他的学习活动与课程安排有关。以下是小华学习活动的逻辑关系:

(1) 如果小华去图书馆学习, 那么他需要带笔记本电脑。

(2) 如果小华有编程作业并且有论文要写, 那么他会去图书馆学习。

(3) 小华有编程作业。

(4) 小华有论文要写。

问题要求:

证明小华需要带笔记本电脑。

2. 环境

开发工具为 Python 3.12, 推荐使用 PyCharm 或 Visual Studio Code 集成开发环境以提高编码效率。

3. 步骤

(1) 题目分析。

根据题意我们定义命题变量：

P ：小华会去图书馆学习。

Q ：小华需要带笔记本电脑。

U ：小华有编程作业。

T ：小华有论文要写。

根据题目给出的逻辑关系，我们可以写出以下命题逻辑表达式：

$P \rightarrow Q$ ：如果小华去图书馆学习，那么他需要带笔记本电脑。

$(U \wedge T) \rightarrow P$ ：如果小华有编程作业并且有论文要写，那么他会去图书馆学习。

U ：小华有编程作业。

T ：小华有论文要写。

现在，我们使用归结原理来证明 Q 。

(2) 将每个命题化为子句形式。

$\neg Q = \neg P \vee Q$

$(U \wedge T) \rightarrow P = \neg U \vee \neg T \vee P$

$U = U$

$T = T$

目标的否定： $\neg Q$ 。

(3) 将所有子句保存进文本文件。

```
~P v Q
~U v ~T v P
U
T
~Q
```

(4) 定义读取子句集文件函数和存放列表。

```
S = [] # 用于存储读取的内容
def read_clause_set(filepath):
    global S
    for line in open(filepath, mode='r', encoding='utf-8'):
        line = line.replace(' ', '').strip()
        line = line.split('v')
        S.append(line)
```

(5) 定义辅助函数。

```
# 取反
def opposite(clause):
    if '~' in clause:
```

```

        return clause.replace('~', '')
    else:
        return '~' + clause

```

(6) 定义归结函数。

```

def resolution():
    global S
    X = S.copy()
    end = False
    while True:
        if end: break
        father = X.pop()
        for i in father[:]:
            if end: break
            for mother in X[:]:
                if end: break
                j = list(filter(lambda x: x == opposite(i), mother))
                if j == []:
                    continue
                else:
                    print('\n亲本子句:' + ' v '.join(father) + ' 和 ' + ' v '.join(mother))
                    father.remove(i)
                    mother.remove(j[0])
                    if (father == [] and mother == []):
                        print('归结式:NIL')
                        end = True
                    elif father == []:
                        print('归结式:' + ' v '.join(mother))
                    elif mother == []:
                        print('归结式:' + ' v '.join(father))
                    else:
                        print('归结式:' + ' v '.join(father) + ' v ' + ' v '.join(mother))

```

(7) 主函数。

```

def main():
    filePath = r'S.txt'
    read_clause_set(filePath)
    resolution()
    result = list(filter(None, S))
    if (result):
        result_list = [item for l in result for item in l]
        print('\n归结结果:' + ' v '.join(result_list))

```

```
else:
    print("\n 完全归结 ")

if __name__ == '__main__':
    main()
```

4. 代码运行结果

亲本子句: $\sim Q$ 和 $\sim P \vee Q$

归结式: $\sim P$

亲本子句: T 和 $\sim U \vee \sim T \vee P$

归结式: $\sim U \vee P$

亲本子句: U 和 $\sim U \vee P$

归结式: P

亲本子句: P 和 $\sim P$

归结式: NIL

完全归结

总 结

本章主要学习和探讨了以下内容：

1. 命题逻辑的基本概念和应用，包括：理解命题联结词的使用，如否定、合取、析取、蕴含和双条件等；学会构建合式公式和识别重言式，并通过 Python 编程解决简单的命题逻辑问题。
2. 谓词逻辑的表达和推理，包括：掌握如何使用量词和谓词来描述更复杂的逻辑关系；学习谓词逻辑与命题逻辑的区别与联系，并解决涉及个体和属性的逻辑推理问题。
3. 归结推理方法的基本原理和应用，包括：掌握归结推理方法，理解其在逻辑推理中的应用；通过 Python 编程解决简单问题，提高逻辑推理能力。

通过这三个案例，读者应该系统地掌握了逻辑推理算法的原理和应用技巧，并能够灵活运用这些算法解决实际复杂逻辑推理问题。这些案例不仅帮助读者建立对逻辑推理算法的直观理解，还使读者在实践中学会算法设计、参数调整以及性能评估等关键知识点的应用。通过项目案例的方式完成具体的问题求解任务，让读者在案例中学会运用这些算法解决实际问题，并在此过程中深入理解逻辑推理的核心概念和方法。

习 题

1. 用 Python 实现专家系统。
2. 完成谓词逻辑归结推理案例。

扩展思考

为深化本章知识理解，建议读者通过以下方面进行延伸学习：

1. 现实问题建模：选择校园生活场景（如课程推荐、实验室资源分配），用一阶谓词逻辑建立知识库，体会逻辑推理在实际问题中的应用价值。
2. 性能对比实验：针对同一推理任务，分别采用 Prolog 内置引擎与 Python 自实现引擎，比较两者的执行效率与内存消耗。

第 4 章

计算智能高级算法

知识导图

视频



计算智能
高级算法



引言

在掌握搜索算法的问题求解逻辑与逻辑推理的形式化方法后，本章将聚焦人工智能的另一核心优化范式——计算智能高级算法，这是解决复杂组合优化、参数寻优问题的关键技术支持。计算智能算法摆脱了传统方法对显式数学模型的依赖，通过模拟自然现象或生物行为的内在规律，实现高效的智能优化。本章将系统拆解三类经典算法：遗传算法模拟生物进化中的自然选择与遗传变异机制，通过编码、选择、交叉、变异等操作在解空间中迭代寻优；蚁群算法借鉴蚂蚁群体的信息素通信与协作觅食行为，利用正反馈机制快速收敛到最优路径；模拟退火算法则源于金属退火的物理过程，通过温度调控与概率性接受准则，有效跳出局部最优解。通过旅行商问题等经典案例的实战演练，你将亲手实现三类算法，对比分析它们在解的最优性、收敛速度及适用场景上的差异，掌握根据问题特性选择优化策略的能力，为后续处理工业调度、参数优化等复杂实际问题奠定坚实的算法基础。

学习目标

- (1) 深入理解遗传算法、蚁群算法与模拟退火算法的核心原理，明确三类算法的灵感来源、核心机制及适用场景边界。
- (2) 掌握三类算法的关键操作与参数设计，包括遗传算法的编码、选择、交叉、变异操作，蚁群算法的信息素更新规则，模拟退火算法的温度调度与状态转移策略。
- (3) 熟练使用 Python 实现三类算法，能够独立完成算法核心模块开发、参数配置与结果验证，适配旅行商问题等组合优化场景。
- (4) 学会对比分析三类算法的性能差异，从解的最优性、收敛速度、时空复杂度角度，为具体问题选择合适的优化策略。
- (5) 具备运用计算智能算法解决实际优化问题的能力，通过实战掌握参数调优技巧，提升算法在复杂场景中的寻优效率。

4.1 理论知识介绍

遗传算法 (genetic algorithm, GA)、蚁群算法 (ant colony optimization, ACO) 和模拟退火算法 (simulated annealing, SA) 都是启发式搜索算法，它们在解决各种优化问题中发挥着重要作用。

4.1.1 遗传算法

遗传算法的首次提出是在1967年，Holland教授的读者Bagley在其博士论文中提到“Genetic Algorithm”，即GA；1975年Holland教授出版了第一本系统叙述其内容的专著，奠定了理论基础。今天，遗传算法已经成为十分流行的算法，在组合优化、自动控制、图像处理、机器学习等领域都有很广泛的运用。

具体的，遗传算法是受自然进化理论启发的一系列搜索算法。通过模仿自然选择和繁殖的过程，遗传算法可以为涉及搜索、优化和学习的各种问题提供高质量的解决方案。同时，它们类似于自然进化，因此可以克服传统搜索和优化算法遇到的一些障碍，尤其是对于具有大量参数和复杂数学表示形式的问题。

1. 类比达尔文进化论

遗传算法是类比自然界的达尔文进化实现的简化版本，在此简要介绍达尔文进化论的原理总结：

- (1) **变异**：种群中单个样本的特征（性状，属性）可能会有所不同，这导致了样本彼此之间有一定程度的差异。
- (2) **遗传**：某些特征可以遗传给其后代。导致后代与双亲样本具有一定程度的相似性。
- (3) **选择**：种群通常在给定的环境中争夺资源。更适应环境的个体在生存方面更具优势，因此会产生更多的后代。

换句话说，进化维持了种群中个体样本彼此不同。那些适应环境的个体更有可能生存，繁殖并将其性状传给下一代。这样，随着世代的更迭，物种变得更加适应其生存环境。而进化的重要推动因素是交叉（crossover）、重组（recombination）或杂交——结合双亲的特征产生后代。交叉有助于维持人口的多样性，并随着时间的推移将更好的特征融合在一起。此外，变异（mutations）或突变（特征的随机变异）可以通过引入偶然性的变化而在进化中发挥重要作用。

2. 遗传算法中的相关概念

遗传算法试图找到给定问题的最佳解。达尔文进化论保留了种群的个体性状，而遗传算法则保留了针对给定问题的候选解集合。这些候选解经过迭代评估（evaluate），用于创建下一代解。更优的解有更大的机会被选择，并将其特征传递给下一代候选解集合。这样，随着代际更新，候选解集合可以更好地解决当前的问题。

(1) **基因型（genotype）**。在自然界中，通过基因型表征繁殖、交叉和突变，基因型是组成染色体的一组基因的集合。在遗传算法中，每个个体都由代表基因集合的染色体构成。例如图4-1中，一条染色体可以表示为二进制串，其中每个位代表一个基因。



图 4-1 类基因图展示

(2) **种群（population）**。遗传算法保持大量的个体（individuals）——针对当前问题的候选解集合。由于每个个体都由染色体表示，因此这些种族的个体可以看作是染色体集合，如图4-2所示。

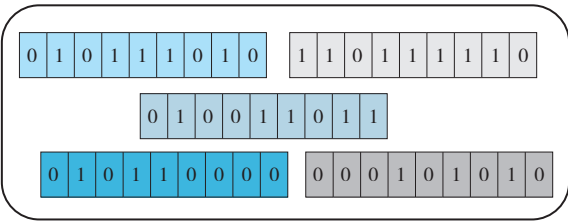


图 4-2 种群基因图展示

(3) **适应度函数（fitness function）**。在算法的每次迭代中，使用适应度函数（也称为目标函数）对个体进行评估。目标函数是用于优化的函数或试图解决的问题。适应度得分更高的个体代表了更好的解，其更有可能被选择繁殖并且其性状会在下一代中得到表现。随着遗传算法的进行，解的质量会提高，适应度会增加，一旦找到具有令人满意的适应度值的解，终止遗传算法。

(4) **选择（selection）**。选择的目的是从当前群体中选出优良的个体，使它们有机会作为父代为下

一代繁衍子孙。根据各个个体的适应度值（适应度值高则被选中的概率高，并非一定能被选中；而适应度值低的也可能被选中，只是被选中的概率较小罢了），按照一定的规则或方法从上一代群体中选出一些优良的个体遗传到下一代种群中。选择的依据是适应性强的个体为下一代贡献一个或多个后代的概率大。

（5）交叉。通过交叉操作可以得到新一代个体，新个体组合了父辈个体的特性（保留了父辈的特征）。将群体中的各个个体随机搭配配对，对每一个个体，以交叉概率交换它们之间的部分染色体，如图4-3所示。

（6）变异。对种群中的每一个个体，以变异概率改变某一个或多个基因座上的基因值为其他的等位基因。同生物界中一样，（与交叉相比）变异发生的概率很低，变异为新个体的产生提供了机会，如图4-4所示。

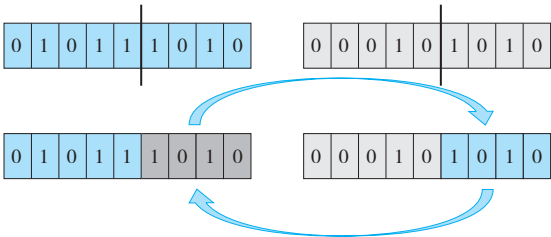


图 4-3 基因交叉过程展示

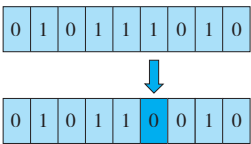


图 4-4 基因变异过程展示

3. 遗传算法总体流程

遗传算法的总体流程如图4-5所示。

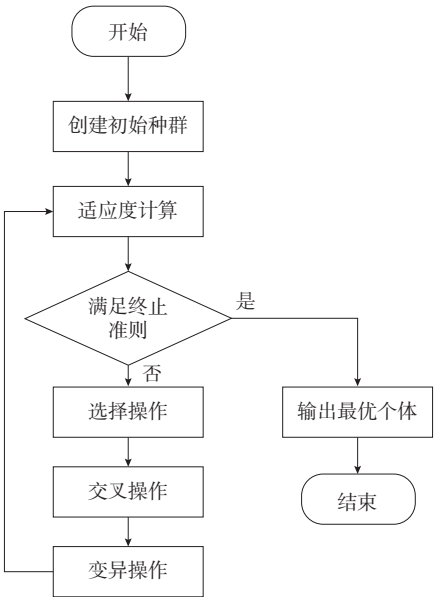


图 4-5 遗传算法流程

（1）初始化：设置进化代数计数器 $g=0$ ，设置最大进化代数 G ，随机生成NP个个体作为初始群 $P(0)$ 。

(2) **个体评价**: 计算群体 $P(t)$ 中各个个体的适应度。

(3) **选择运算**: 将选择算子作用于群体, 根据个体的适应度, 按照一定的规则或方法, 选择一些优良个体遗传到下一代群体。

(4) **交叉运算**: 将交叉算子作用于群体, 对选中的成对个体, 以某一概率交换它们的之间的染色体, 产生新个体。

(5) **变异运算**: 将变异算子作用于群体, 对选中的个体, 以某一概率改变某一个或某一些基因值为其他的等位基因。群体 $P(t)$ 经过选择、交叉和变异运算之后得到下一代群体 $P(t+1)$, 计算适用值, 并根据适应值进行排序, 准备进行下一次遗传操作。

(6) **终止条件判断**: 若 $g < G$, 则 $g = g + 1$, 转到步骤 (2), 否则, 进化过程中所得到的具有最大适应度的个体作为最优解输出, 终止计算。

4. 关键参数

(1) **群体规模**。群体规模将影响遗传优化的最终结果以及遗传算法的执行效率。当群体规模 NP 太小时, 遗传优化性能一般不会太好。采用较大的群体规模可以减小遗传算法陷入局部最优解的机会, 但较大的群体规模意味着计算复杂度较高。一般 NP 取 10 ~ 200。

(2) **交叉概率**。交叉概率 P_c 控制着交叉操作被使用的频度。较大的交叉概率可以增强遗传算法开辟新的搜索区域的能力, 但高性能的模式遭到破坏的可能性增大; 若交叉概率太低, 遗传算法搜索可能陷入迟钝状态。一般 P_c 取 0.25 ~ 1.00。

(3) **变异概率**。变异在遗传算法中属于辅助性的搜索操作, 它的主要目的是保持群体的多样性。一般低频度的变异可防止群体中重要基因的可能丢失, 高频度的变异将使遗传算法趋于纯粹的随机搜索。通常 P_m 取 0.001 ~ 0.1。

(4) **进化代数**。终止进化代数 G 是表示遗传算法运行结束条件的一个参数, 它表示遗传算法运行到指定的进化代数之后就停止运行, 并将当前群体中的最佳个体作为所求问题的最优解输出。一般视具体问题而定, G 取 100 ~ 1 000。

5. 应用范围

(1) **旅行商问题 (traveling salesman problem)**: 旅行商问题是一个经典的组合优化问题, 目标是找到一条路径, 使得旅行商能够访问所有城市并返回起始城市, 同时总路线最短。遗传算法可以用来寻找近似最优解。

(2) **机器调度问题 (machine scheduling problem)**: 机器调度问题涉及将若干任务分配给多台机器, 并确定任务的执行顺序和时间, 以最大化生产效率或最小化完成时间。遗传算法可以用来优化任务的分配和调度策略。

(3) **参数优化问题 (parameter optimization)**: 在许多科学和工程领域中, 需要通过调整模型或算法中的参数来优化性能指标。遗传算法可以用来搜索参数空间, 找到最优的参数组合, 从而优化模型或算法的性能。

(4) **神经网络训练 (neural network training)**: 神经网络的训练通常涉及调整网络的权重和偏置, 以最小化损失函数。遗传算法可以用来搜索神经网络的参数空间, 寻找最优的权重和偏置组合, 从而提高网络的性能。

(5) **组合优化问题 (combinatorial optimization)**: 组合优化问题涉及在给定约束条件下, 从大量组合中找到最优解。例如, 图的着色问题、背包问题等。遗传算法可以用来搜索组合空间, 找到满足约束

条件并具有最优特性的组合。

4.1.2 蚁群算法

蚁群系统 (ant system或ant colony system) 是由意大利学者Dorigo、Maniezzo等人于20世纪90年代首先提出来的。他们在研究蚂蚁觅食的过程中,发现单个蚂蚁的行为比较简单,但是蚁群整体却可以体现一些智能的行为。例如蚁群可以在不同的环境下,寻找最短到达食物源的路径。这是因为蚁群内的蚂蚁可以通过某种信息机制实现信息的传递。后又经进一步研究发现,蚂蚁会在其经过的路径上释放一种可以称之为“信息素”的物质,蚁群内的蚂蚁对“信息素”具有感知能力,它们会沿着“信息素”浓度较高路径行走,而每只路过的蚂蚁都会在路上留下“信息素”,这就形成一种类似正反馈的机制,这样经过一段时间后,整个蚁群就会沿着最短路径到达食物源了。

将蚁群算法应用于解决优化问题的基本思路为:用蚂蚁的行走路径表示待优化问题的可行解,整个蚂蚁群体的所有路径构成待优化问题的解空间。路径较短的蚂蚁释放的信息素量较多,随着时间的推进,较短的路径上累积的信息素浓度逐渐增高,选择该路径的蚂蚁个数也愈来愈多。最终,整个蚂蚁群体在正反馈的作用下集中到最佳的路径上,此时对应的便是待优化问题的最优解。

1. 蚁群算法特点

(1) 并行。每只蚂蚁的搜索过程相互独立,所以蚁群算法可以看作一个分布式的多智能体系统,它在问题空间的多点同时开始独立的解搜索,不仅增加了算法的可靠性,也使得算法具有较强的全局搜索能力。

(2) 自组织。所谓的自组织就是没有外界干预,系统从无序到有序的变化过程。

(3) 鲁棒性。蚁群算法对初始路线的要求不高,即蚁群算法的求解结果不依赖于初始路线的选择,而且在搜索过程中不需要进行人工的调整。此外,蚁群算法的参数较少,设置简单,因而该算法易于应用到组合优化问题的求解。

(4) 正反馈。这一特点是蚁群算法的重要特征,它使得算法进化过程得以进行。体现在,蚂蚁能够最终找到最优路径,直接依赖于其在路径上信息素的堆积,而信息素的堆积是一个正反馈的过程。

2. 蚁群算法基本原理

在算法的初始时刻,将 m 只蚂蚁随机地放到 n 座城市,同时,将每只蚂蚁的禁忌表 tabu 的第一个元素设置为它当前所在的城市。此时各路径上的信息素量相等,设 $\tau_{ij}(0)=c$ (c 为一较小的常数),在时刻 t ,蚂蚁 k 从城市 i 转移到城市 j 的概率为

$$p_{ij}^k(t) = \begin{cases} \frac{[\tau_{ij}(t)]^\alpha \cdot [\eta_{ij}(t)]^\beta}{\sum_{s \in J_k(i)} [\tau_{is}(t)]^\alpha \cdot [\eta_{is}]^\beta}, & j \in J_k(i) \\ 0, & \text{Others} \end{cases} \quad (4-1)$$

式(4-1)中, $J_k(i)=\{1, 2, \dots, n\}$ 。 tabu_k 表示蚂蚁 k 下一步允许选择的城市集合。禁忌表 tabu_k 记录了蚂蚁 k 当前走过的城市。当所有 n 座城市都加入禁忌表中时,蚂蚁 k 便完成了一次周游,此时蚂蚁 k 所走过的路径便是TSP问题的一个可行解。

式(4-1)中, η_{ij} 是一个启发式因子,表示蚂蚁从城市 i 转移到城市 j 的期望程度。在蚁群算法中, η_{ij} 通常取城市 i 与城市 j 之间距离的倒数。 α 和 β 分别表示信息素和期望启发式因子的相对重要程度。当所有蚂蚁完成一次周游后,各路径上的信息素根据下式更新:

$$\tau_{ij}(t+n) = (1-\rho) \cdot \tau_{ij}(t) + \Delta \tau_{ij} \tag{4-2}$$

式(4-2)中， ρ ($0 < \rho < 1$)表示路径上信息素蒸发系数， $1-\rho$ 表示信息素的持久性系数； $\Delta \tau_{ij}$ 表示本次迭代中边 ij 上信息素的增量，即：

$$\Delta \tau_{ij} = \sum_{k=1}^m \Delta \tau_{ij}^k \tag{4-3}$$

式(4-3)中， $\Delta \tau_{ij}^k$ 表示第 k 只蚂蚁在本次迭代中留在边 ij 上的信息素量，如果蚂蚁 k 没有经过边 ij ，则 $\Delta \tau_{ij}^k=0$ ， $\Delta \tau_{ij}^k$ 可以表示为

$$\Delta \tau_{ij}^k = \begin{cases} \frac{Q}{L_k}, & \text{蚂蚁 } k \text{ 在本次周游时经过边 } ij \text{ 时} \\ 0, & \text{Others} \end{cases} \tag{4-4}$$

式(4-4)中， Q 为正常数； L_k 表示第 k 只蚂蚁在本次周游中所走过路径的长度。

3. 蚁群算法总体流程

蚁群算法的总体流程如图4-6所示。

(1) 初始化参数。在计算之初，需对相关的参数进行初始化，如蚁群规模（蚂蚁数量） m 、信息素重要程度因子 α 、启发函数重要程度因子 β 、信息素蒸发系数 ρ 、信息素释放总量 Q 、最大迭代次数 $iter_max$ 、迭代次数初值 $iter=1$ 。

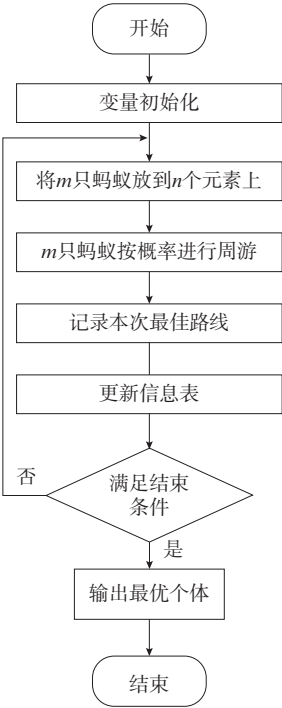


图 4-6 蚁群算法流程

(2) 构建解空间。将各个蚂蚁随机地置于不同出发点，对每个蚂蚁 k ($k=1, 2, \dots, m$)，计算其下一个待访问的城市，直到所有蚂蚁访问完所有的城市。

(3) 更新信息素。计算各个蚂蚁经过的路径长度 $L_k(1, 2, \cdots, m)$ ，记录当前迭代次数中的最优解（最短路径）。同时对各个城市连接路径上的信息素浓度进行更新。

(4) 判断是否终止。若 $iter < iter_max$ ，则清空蚂蚁经过路径的记录表，并返回步骤（2）；否则，终止计算，输出最优解。

4. 蚁群算法关键参数

蚁群算法关系参数见表4-1。

表 4-1 蚁群算法关键参数

关键参数	设 置	原 因
信息素启发式因子 α	[1, 4]	α 的大小反映了蚁群在路径搜索中随机性因素作用的强度，其值越大，蚂蚁选择以前走过的路径的可能性就越大，搜索的随机性就会减弱；而当启发式因子 α 的值过小时，则易使蚁群的搜索过早陷于局部最优
期望启发因子 β	[3, 5]	反映蚁群在搜索最优路径的过程中的先验性和确定性因素的作用强度。期望启发因子 β 的值越大，蚂蚁在某个局部点上选择局部最短路径的可能性就越大，虽然这个时候算法的收敛速度得以加快，但蚁群搜索最优路径的随机性减弱，而此时搜索易于陷入局部最优解
信息素蒸发系数 ρ	[0, 1]	ρ 表示信息素蒸发系数， $1-\rho$ 则表示信息素持久性系数。因此， ρ 的取值范围应该是 0~1 之间的一个数，表示信息素的蒸发程度，它实际上反映了蚂蚁群体中个体之间相互影响的强弱。 ρ 过小时，则表示以前搜索过的路径被再次选择的可能性过大，会影响到算法的随机性能和全局搜索能力； ρ 过大时，说明路径上的信息素挥发的相对变多，虽然可以提高算法的随机搜索性能和全局搜索能力，但过多无用搜索操作势必会降低算法的收敛速度
蚂蚁数目 m	任意	子集增大（即蚂蚁数量增多），可以提高蚁群算法的全局搜索能力以及算法的稳定性；但蚂蚁数目增大后，会使大量的曾被搜索过的解（路径）上的信息素的变化趋于平均，信息正反馈的作用不明显，虽然搜索的随机性得到了加强，但收敛速度减慢；反之，子集较小（蚂蚁数量少），特别是当要处理的问题规模比较大时，会使那些从来未被搜索到的解（路径）上的信息素减小到接近于 0，搜索的随机性减弱，虽然收敛速度加快了，但会使算法的全局性能降低，算法的稳定性差，容易出现过早停滞现象
信息素强度 Q	任意	对算法性能起着主要作用的是信息启发式因子 α 、期望启发式因子 β 和信息素挥发因子 ρ 这三个参数，总信息量 Q 对算法性能的影响有赖于上述三个参数的选取，以及算法模型的选取。例如，在 ant-cycle 模型和 ant-quantity 模型中，总信息量 Q 所起的作用显然是有很大差异的，即随着问题规模的不同，其影响程度也将不同。相关人员研究结果表明：总信息量 Q 对 ant-cycle 模型蚁群算法的性能没有明显的影响。因此，在算法参数的选择上，参数 Q 不必作特别的考虑，可以任意选取
最大进化代数 G	任意	蚁群算法运行到指定的进化代数之后就停止运行，并将当前群体中的最佳个体作为所求问题的最优解输出

5. 蚁群算法应用范围

该算法应用于其他组合优化问题，如旅行商问题、指派问题、Job-shop调度问题、车辆路由问题、图着色问题和网络路由问题等。最近几年，该算法在网络路由中的应用受到越来越多学者的关注，并提出了一些新的基于蚂蚁算法的路由算法。同传统的路由算法相比较，该算法在网络路由中具有信息分布式性、动态性、随机性和异步性等特点，而这些特点正好能满足网络路由的需要。

4.1.3 模拟退火算法

模拟退火算法（simulated annealing algorithm）来源于固体退火原理，是一种基于概率的算法。将固体加温至充分高的温度，再让其徐徐冷却，加温时，固体内部粒子随温升变为无序状，内能增大，分子和原子越不稳定。而徐徐冷却时粒子渐趋有序，能量减少，原子越稳定。在冷却（降温）过程中，固

体在每个温度都达到平衡态，最后在常温时达到基态，内能减为最小。模拟退火算法从某一较高初温出发，伴随温度参数的不断下降，结合概率突跳特性在解空间中随机寻找目标函数的全局最优解，即在局部最优解能概率性地跳出并最终趋于全局最优。模拟退火算法是通过赋予搜索过程一种时变且最终趋于零的概率突跳性，从而可有效避免陷入局部极小并最终趋于全局最优的串行结构的优化算法。

1. 物理退火过程

模拟退火算法的核心思想与热力学的原理极为类似。在高温下，液体的大量分子彼此之间进行着相对自由移动。如果该液体慢慢地冷却，热能原子可动性就会消失。大量原子常常能够自行排列成行，形成一个纯净的晶体，该晶体在各个方向上都被完全有序地排列在几百万倍于单个原子的距离之内。对于这个系统来说，晶体状态是能量最低状态，而所有缓慢冷却的系统都可以自然达到这个最低能量状态。若液态金属被迅速冷却，则不一定会达到这一状态，而只能达到一种只有较高能量的多晶体状态或非结晶状态。因此，这一过程的本质在于缓慢地冷却，以争取足够的时间，让大量原子在丧失可动性之前进行重新分布，这是确保能量达到低能量状态所必需的条件。简单而言，物理退火过程由以下几部分组成：加温过程、等温过程和冷却过程。

（1）**加温过程**。其目的是增强粒子的热运动，使其偏离平衡位置。当温度足够高时，固体将熔解为液体，从而消除系统原先可能存在的非均匀态，使随后进行的冷却过程以某一平衡态为起点。熔解过程与系统的能量增大过程相联系，系统能量也随温度的升高而增大。

（2）**等温过程**。通过物理学的知识得知，对于与周围环境交换热量而温度不变的封闭系统，系统状态的自发变化总是朝着自由能减小的方向进行；当自由能达到最小时，系统达到平衡态。

（3）**冷却过程**。其目的是使粒子的热运动减弱并逐渐趋于有序，系统能量逐渐下降，从而得到低能量的晶体结构。

模拟退火与物理退火算法关系见表4-2。

表 4-2 模拟退火与物理退火算法关系表

物理退火	模拟退火
粒子状态	解
能量最低态	最优解
熔解过程	设定初温
等温过程	Metropolis 采样过程
冷却	控制参数的下降
能量	目标函数

根据Metropolis准则，粒子在温度 T 时趋于平衡的概率为 $\exp(-\Delta E/T)$ ，其中 E 为温度 T 时的内能， ΔE 为其改变量。用固体退火模拟组合优化问题，将内能 E 模拟为目标函数值，温度 T 演化成控制参数，即得到解组合优化问题的模拟退火算法：由初始解 X_0 和控制参数初值 T 开始，对当前解重复“产生新解→计算目标函数差→接受或舍弃”的迭代，并逐步减小 T 值，算法终止时的当前解即为所得近似最优解，这是基于蒙特卡洛迭代求解法的一种启发式随机搜索过程。退火过程由冷却进度表控制，包括控制参数的初值 T_0 及其衰减因子 K 、每个 T 值时的迭代次数 L 和停止条件。

2. 模拟退火算法特点

（1）**以一定的概率接受恶化解**。模拟退火算法在搜索策略上不仅引入了适当的随机因素，而且还

引入了物理系统退火过程的自然机理。这种自然机理的引入,使模拟退火算法在迭代过程中不仅接受使目标函数值变“好”的点,而且还能够以一定的概率接受使目标函数值变“差”的点。迭代过程中出现的状态是随机产生的,并且不强求后一状态一定优于前一状态,接受概率随着温度的下降而逐渐减小。很多传统的优化算法往往是确定性的,从一个搜索点到另一个搜索点的转移有确定的转移方法和转移关系,这种确定性往往可能使得搜索点远达不到最优点,因而限制了算法的应用范围。而模拟退火算法以一种概率的方式来进行搜索,增加了搜索过程的灵活性。

(2) **引进算法控制参数**。引进类似于退火温度的算法控制参数,它将优化过程分成若干阶段,并决定各个阶段下随机状态的取舍标准,接受函数由Metropolis算法给出一个简单的数学模型。模拟退火算法有两个重要的步骤:一是在每个控制参数下,由前迭代点出发,产生邻近的随机状态,由控制参数确定的接受准则决定此新状态的取舍,并由此形成一定长度的随机马尔可夫链;二是缓慢降低控制参数,提高接受准则,直至控制参数趋于零,状态链稳定于优化问题的最优状态,从而提高模拟退火算法全局最优解的可靠性。

(3) **对目标函数要求少**。传统搜索算法不仅需要利用目标函数值,而且往往需要目标函数的导数值等其他一些辅助信息才能确定搜索方向;当这些信息不存在时,算法就失效了。而模拟退火算法不需要其他的辅助信息,而只是定义邻域结构,在其邻域结构内选取相邻解,再用目标函数进行评估。

3. 模拟退火算法基本原理

模拟退火的主要思想是:在搜索区间随机游走(即随机选择点),再利用Metropolis抽样准则,使随机游走逐渐收敛于局部最优解。而温度是Metropolis算法中的一个重要控制参数,可以认为这个参数的大小控制了随机过程向局部或全局最优解移动的快慢。

Metropolis 是一种有效的重点抽样法,其算法为:系统从一个能量状态变化到另一个状态时,相应的能量从 E_1 变化到 E_2 ,其概率为

$$p = \exp\left(-\frac{E_2 - E_1}{T}\right) \quad (4-5)$$

式(4-5)中,如果 $E_2 < E_1$,系统接受此状态;否则,以一个随机的概率接受或丢弃此状态。状态2被接受的概率为

$$p(1 \rightarrow 2) = \begin{cases} 1, & E_2 < E_1 \\ \exp\left(-\frac{E_2 - E_1}{T}\right), & E_2 \geq E_1 \end{cases} \quad (4-6)$$

这样经过一定次数的迭代,系统会逐渐趋于一个稳定的分布状态。终点抽样时,新状态下如果向下,则接受(局部最优);若向上(全局搜索),则以一定概率接受。模拟退火算法从某个初始解出发,经过大量解的变换后,可以求得给定控制参数值时组合优化问题的相对最优解。然后减小控制参数 T 的值,重复执行Metropolis算法,就可以在控制参数 T 趋于零时,最终求得组合优化问题的整体最优解。控制参数的值必须缓慢衰减。

温度是Metropolis算法的一个重要控制参数,模拟退火可视为递减控制参数 T 时Metropolis算法的迭代。开始时 T 值大,可以接受较差的恶化解;随着 T 的减小,只能接受较好的恶化解;最后在 T 趋于0时,就不再接受任何恶化解了。在无限高温时,系统立即均匀分布,接受所有提出的变换。 T 的衰减越小, T 到达终点的时间越长;但可使马尔可夫链减小,以使到达准平衡分布的时间变短。

4. 模拟退火算法总体流程

模拟退火算法新解的产生和接受可分为如下三个步骤：

(1) 由一个产生函数从当前解产生一个位于解空间的新解；为便于后续的计算和接受，减少算法耗时，通常选择由当前解经过简单变换即可产生新解的方法。注意，产生新解的变换方法决定了当前新解的邻域结构，因而对冷却进度表的选取有一定的影响。

(2) 判断新解是否被接受，判断的依据是一个接受准则，最常用的接受准则是 Metropolis 准则：若 $\Delta E < 0$ ，则接受 X' 作为新的当前解 X ；否则，以概率 $\exp(-\Delta E/T)$ 接受 X' 作为新的当前解 X 。

(3) 当新解被确定接受时，用新解代替当前解，这只需将当前解中对应于产生新解时的变换部分予以实现，同时修正目标函数值即可。此时，当前解实现了一次迭代，可在此基础上开始下一轮试验。若当新解被判定为舍弃，则在原当前解的基础上继续下一轮试验。模拟退火算法求得的解与初始解状态（算法迭代的起点）无关，具有渐近收敛性。模拟退火算法可以分解为解空间、目标函数和初始解三部分。该算法具体流程如图4-7所示。

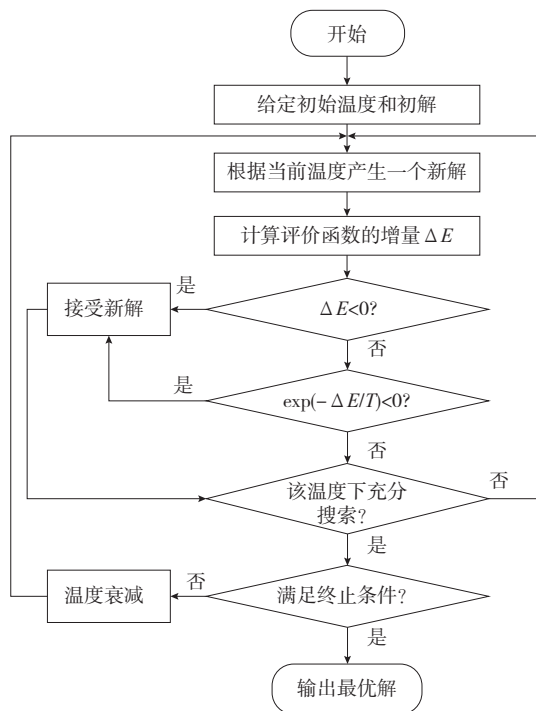


图 4-7 模拟退火算法流程

- ① 初始化：设置初始温度 T_0 （充分大）、初始解状态 X_0 （是算法迭代的起点）、每个 T 值的迭代次数 L ；
- ② 对 $k=1, \dots, L$ 做第③至第⑥步；
- ③ 产生新解 X' ；
- ④ 计算增量 $\Delta E = E(X') - E(X)$ ，其中 $E(X)$ 为评价函数；
- ⑤ 若 $\Delta E < 0$ ，则接受 X' 作为新的当前解，否则以概率 $\exp(-\Delta E/T)$ 接受 X' 作为新的当前解；
- ⑥ 如果满足终止条件，则输出当前解作为最优解，结束程序；
- ⑦ T 逐渐减小，且 $T \rightarrow 0$ ，然后转第②步。

5. 模拟退火算法关键参数

为了得到最优解，该算法通常要求较高的初温以及足够多次的抽样，这使算法的优化时间往往过长。从算法结构知，新的状态产生函数、初温、退温函数、马尔可夫链长度和算法停止准则，是直接影响算法优化结果的主要环节。模拟退火算法关键参数见表4-3。

表 4-3 模拟退火算法关键参数表

关键参数	取 值	原 因
状态产生函数	\	一般情况下状态产生函数由两部分组成，即产生候选解的方式和产生候选解的概率分布。候选解的产生方式由问题的性质决定，通常在当前状态的邻域结构内以一定概率产生
初温	\	温度 T 在算法中具有决定性的作用，它直接控制着退火的走向。由随机移动的接受准则可知：初温越大，获得高质量解的几率就越大，且 Metropolis 的接收率约为 1。然而，初温过高会使计算时间增加。为此，可以均匀抽样一组状态，以各状态目标值的方差为初温
退温函数	\	退温函数即温度更新函数，用于在外循环中修改温度值。目前，最常用的温度更新函数为指数退温函数，即 $T(n+1)=K \times T(n)$ ，其中 $0<K<1$ 是一个非常接近于 1 的常数
马尔可夫链长度 L	一般 L 取 100~1 000	马尔可夫链长度是在等温条件下进行迭代优化的次数，其选取原则是在衰减参数 T 的衰减函数已选定的前提下， L 应选得在控制参数的每一取值上都能恢复准平衡
算法停止准则	\	算法停止准则用于决定算法何时结束。可以简单地设置温度终值 T_f ，当 $T=T_f$ 时算法终止。然而，模拟退火算法的收敛性理论中要求 T_f 趋向于零，这其实是不实际的。常用的停止准则包：设置终止温度的阈值，设置迭代次数阈值，或者当搜索到的最优值连续保持不变时停止搜索

6. 模拟退火算法应用范围

(1) 旅行商问题 (TSP): TSP是一个著名的组合优化问题，模拟退火算法可以在搜索空间中寻找城市间的最短路径，以实现旅行商的最优路径规划。

(2) 图像分割: 模拟退火算法可用于图像分割，通过优化分割阈值，将图像分成不同的区域，以便进一步处理或分析。

(3) 参数优化: 在机器学习和深度学习中，模拟退火算法可以用于优化模型的超参数，如学习率、层数、神经元数量等。

(4) 物理模拟: 模拟退火算法可以模拟固体材料的晶格结构，优化分子模型以获得稳定的能量最低态。

4.2 案例描述

4.2.1 案例背景介绍

现实世界中的优化问题往往具有高度复杂性，如物流路径规划、生产排程和机器学习超参数调优等。这些问题的解空间通常规模庞大且非线性，传统优化方法难以有效应对。遗传算法、蚁群算法和模拟退火算法作为三类典型的智能优化方法，分别借鉴了生物进化、群体智能和物理退火过程的自然机理，能够有效处理这类复杂优化问题。通过本案例，读者将理解这些算法如何突破传统方法的局限，在合理时间内获得近似最优解，从而提升实际业务决策的效率与质量。

4.2.2 案例技术分析

为了解决上述提到的优化问题，需要掌握以下知识和技术：

遗传算法：理解基因编码、选择、交叉和变异等基本操作，以及如何根据问题特性设计适应度函数。

蚁群算法：学习蚂蚁如何通过信息素进行间接通信，并利用这一机制寻找最短路径。

模拟退火算法：掌握如何模拟物质加热后再缓慢冷却的过程，并通过概率性地接受解的方式来跳出局部最优，以达到全局优化。

相关的技术路线和思路包括：

问题建模：将实际问题抽象成算法可以处理的数学模型。

算法设计与实现：根据问题特点选择合适的算法，并调整算法参数以适应特定的问题实例。

性能评估：通过对比不同的算法或参数设置下的运行结果，评价算法的有效性和效率。

4.3 设计与实现

4.3.1 遗传算方案例

1. 案例任务

在本次遗传算方案例中，我们将专注于解决著名的旅行商问题。TSP问题是组合优化中的一个经典问题，它要求找到一条最短的可能路径，以便旅行商访问一系列城市各一次并返回到起始城市。我们将采用遗传算法这一人工智能技术来寻求问题的近似最优解，并通过Python编程语言实现整个求解过程。通过本案例，读者将能够体会到遗传算法如何在全局搜索中寻找优秀的解决方案，并了解其背后的原理和实现技巧。

2. 环境

开发工具为Python 3.10，推荐使用PyCharm或Visual Studio Code集成开发环境以提高编码效率。

相关库：random（用于生成随机数）。

3. 步骤

（1）导入random包。

```
import random
```

（2）定义城市列表和距离矩阵。

定义的旅行商问题带权图如图4-8所示。

```
city_list = ['A', 'B', 'C', 'D', 'E']
distance_matrix = {
    'A': {'A': 0, 'B': 2, 'C': 9, 'D': 10, 'E': 5},
    'B': {'A': 2, 'B': 0, 'C': 6, 'D': 4, 'E': 8},
    'C': {'A': 9, 'B': 6, 'C': 0, 'D': 3, 'E': 7},
    'D': {'A': 10, 'B': 4, 'C': 3, 'D': 0, 'E': 6},
    'E': {'A': 5, 'B': 8, 'C': 7, 'D': 6, 'E': 0}
}
```

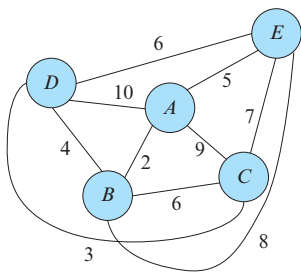


图 4-8 旅行商问题带权图

(3) 遗传算法参数设置。

```
population_size = 50 # 种群数量
elite_size = 10 # 选择个体数量
mutation_rate = 0.01 # 突变概率
generations = 100 # 迭代次数
```

(4) 创建初始种群。

```
# 创建一个随机个体 (路径)
def create_individual(city_list):
    individual = random.sample(city_list, len(city_list))
    return individual

def create_population(city_list, population_size):
    population = []
    for _ in range(population_size):
        individual = create_individual(city_list)
        population.append(individual)
    return population
```

(5) 选择精英个体。

```
def select_elite(population, elite_size):
    population_fitness = [(individual, calculate_fitness(individual)) for
        individual in population]
    population_fitness = sorted(population_fitness, key=lambda x: x[1])
    elite = [individual for individual, _ in population_fitness[:elite_size]]
    return elite
```

(6) 交叉操作。

```
def crossover(parent1, parent2):
    child = [None] * len(parent1)
    geneA = random.randint(0, len(parent1)-1)
    geneB = random.randint(0, len(parent1)-1)
    start_gene = min(geneA, geneB)
    end_gene = max(geneA, geneB)
    for i in range(start_gene, end_gene+1):
        child[i] = parent1[i]

    for i in range(len(parent2)):
        if parent2[i] not in child:
            for j in range(len(child)):
                if child[j] is None:
                    child[j] = parent2[i]
                    break

    return child
```

(7) 变异操作。

```
def mutate(individual):
    for i in range(len(individual)):
        if random.random() < mutation_rate:
            j = random.randint(0, len(individual)-1)
            individual[i], individual[j] = individual[j], individual[i]
    return individual
```

(8) 进化步骤。

```
def evolve_population(population, elite_size):
    elite = select_elite(population, elite_size)
    population_size = len(population)
    children = []

    while len(children) < population_size:
        parent1 = random.choice(elite)
        parent2 = random.choice(elite)
        child = crossover(parent1, parent2)
        child = mutate(child)
        children.append(child)
    return children
```

(9) 适应度函数。

```
def calculate_fitness(individual):
    total_distance = 0
    for i in range(len(individual)-1):
        city1 = individual[i]
        city2 = individual[i+1]
        total_distance += distance_matrix[city1][city2]
    return total_distance
```

(10) 主函数。

```
def main():
    population = create_population(city_list, population_size)

    for i in range(generations):
        population = evolve_population(population, elite_size)

    best_individual = min(population, key=calculate_fitness)
    best_distance = calculate_fitness(best_individual)

    print("最佳路径:", best_individual)
```

```
print(" 最短距离:", best_distance)

if __name__ == '__main__':
    main()
```

4. 代码运行结果

```
最佳路径: ['C', 'D', 'B', 'A', 'E']
最短距离: 13
```

4.3.2 蚁群算法案例

1. 案例任务

本案例旨在利用蚁群算法解决旅行商问题。我们将使用Python编程语言实现蚁群算法，并应用于解决一个具体的旅行商问题。通过该案例，读者将学习如何设计和实现蚁群算法，并将其应用于一个经典的组合优化问题。案例的目标是找到旅行商问题的最优解或近似最优解，以实现最短的旅行路径和最小的旅行成本。

2. 环境

开发工具为 Python 3.10，推荐使用 PyCharm 或 Visual Studio Code 集成开发环境以提高编码效率。

相关库：NumPy（用于处理数值计算）、Random（用于随机生成蚂蚁）。

3. 步骤

（1）导入包。

```
import random
import numpy as np
```

（2）定义城市坐标。

```
city_location = {
    'A': (5, 10),
    'B': (6, 15),
    'C': (10, 15),
    'D': (14, 14),
    'E': (20, 10),
    'F': (16, 5),
    'G': (8, 5),
    'H': (4, 8),
    'I': (8, 12),
    'J': (12, 12)
}
```

（3）将数据存储在 Graph 中，并且计算各个城市之间的距离 distances。

```
class Graph(object):
    def __init__(self, city_location, pheromone=1.0, alpha=1.0, beta=1.0):
        self.city_location = city_location
        self.n = len(city_location)
```

```

        # phernmone 矩阵
        self.pheromone = [[pheromone for _ in range(self.n)] for _ in range(self.n)]
        self.alpha = alpha
        self.beta = beta
        # 城市之间的距离矩阵
        self.distances = self._generate_distances()

# 欧氏距离作为图的权重
def _generate_distances(self):
    distances = []
    # 遍历行
    # 遍历列
    # 如果行值 = 列, 比如 (1,1) (2,2), 这样就是相同的城市, 则距离为零
    # 如果行值 < 列, 比如 (1,2) (2,3), 这样就是不同的城市, 距离按欧氏距离公式来算
    # 否则, 将矩阵的斜对称位置上的距离来补 (原因: 城市之间距离矩阵是一个 distances[i]
[j] = distances[j][i] 特点的矩阵, 这样可减少至少一半的计算)
    for index_i, _i in enumerate(self.city_location):
        row = []
        for index_j, _j in enumerate(self.city_location):
            if _i == _j:
                row.append(0)
            elif _i < _j:
                x1, y1 = self.city_location[str(_i)]
                x2, y2 = self.city_location[str(_j)]
                row.append(np.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2))
            else:
                row.append(distances[index_j][index_i])
        distances.append(row)
    return distances

```

(4) 初始化蚂蚁种群, 我们将各项能力赋予蚂蚁。

```

class Ant(object):
    def __init__(self, graph, start_city):
        self.graph = graph
        self.start_city = start_city
        self.curr_city = start_city
        self.visited = [False for _ in range(graph.n)]
        self.visited[start_city] = True
        self.tour_length = 0
        self.tour = [start_city]

    def choose_next_city(self):

```

```

        # calculate probabilities of all cities
        probs = []
        total_prob = 0
        for i in range(self.graph.n):
            if not self.visited[i]:
                prob = self.graph.pheromone[self.curr_city][i] ** self.graph.alpha * \
(1.0 / self.graph.distances[self.curr_city][i]) ** self.graph.beta
                probs.append((i, prob))
                total_prob += prob
        # select the next city randomly based on the probabilities
        r = random.uniform(0, total_prob)
        upto = 0
        for i, prob in probs:
            if upto + prob >= r:
                self.curr_city = i
                self.visited[i] = True
                self.tour.append(i)
                self.tour_length += self.graph.distances[self.tour[-2]][i]
                return
            upto += prob

    def tour_complete(self):
        return len(self.tour) == self.graph.n

```

(5) 在每个迭代周期结束后进行更新，即在所有蚂蚁完成当前迭代周期后，根据其路径长度和信息素更新信息素浓度。

```

def ant_colony(graph, num_ants, num_iterations, evaporation_rate=0.5, q=500):
    shortest_tour = None # 最短路径
    shortest_tour_length = float('inf') # 最短路径长度
    for i in range(num_iterations):
        ants = [Ant(graph, random.randint(0, graph.n - 1)) for _ in range(num_ants)]
        # 随机生成蚂蚁
        for ant in ants:
            while not ant.tour_complete():
                ant.choose_next_city() # 选择下一个城市
                ant.tour_length += graph.distances[ant.tour[-1]][ant.start_city] # 计算
路径长度
            if ant.tour_length < shortest_tour_length:
                shortest_tour_length = ant.tour_length
                shortest_tour = ant.tour[:]
        for i in range(graph.n):
            for j in range(graph.n):

```

```
        if i != j:
            graph.pheromone[i][j] *= (1 - evaporation_rate) # 更新挥发信息素
        for ant in ants:
            if (i, j) in zip(ant.tour, ant.tour[1:]):
                graph.pheromone[i][j] += q / ant.tour_length # 更新释放
信息素
    return shortest_tour, shortest_tour_length
```

(6) 主函数。

```
if __name__ == '__main__':

    alpha=1.0 # 信息素启发式因子
    beta=3.0 # 期望启发因子
    evaporation_rate=0.5 # 信息素蒸发系数
    q=500 # 信息素强度
    num_ants=500 # 蚂蚁数目
    num_iterations=1000 # 最大进化代数

    g = Graph(city_location)
    shortest_tour, shortest_tour_length = ant_colony(graph=g, num_ants=500, num_iterations=1000)
    # 城市索引
    city_index = {city: i for i, city in enumerate(sorted(city_location))}
    # 城市名称列表
    shortest_tour = [list(city_index.keys())[i] for i in shortest_tour]
    # 打印结果
    print(" 最优路径为 ", shortest_tour)
    print(" 最优路径总距离为 ", shortest_tour_length)
```

4. 代码运行结果

```
最优路径为 ['B', 'I', 'A', 'H', 'G', 'F', 'E', 'D', 'J', 'C']
最优路径总距离为 46.49537571699877
```

4.3.3 模拟退火算法案例

1. 案例任务

本案例旨在利用模拟退火算法解决旅行商问题。我们将使用Python编程语言实现模拟退火算法，并应用于解决一个具体的旅行商问题。通过该案例，读者将学习如何设计和实现模拟退火算法，并将其应用于一个经典的组合优化问题。案例的目标是找到旅行商问题的最优解或近似最优解，以实现最短的旅行路径和最小的旅行成本。

2. 环境

开发工具为 Python 3.10，推荐使用 PyCharm 或 Visual Studio Code 集成开发环境以提高编码效率。

相关库：NumPy（用于处理数值计算）、Typing（用于构建 Dict、List、Tuple 数据结构）。

3. 步骤

(1) 导入包。

```
from typing import Dict, List, Tuple
import numpy as np
```

(2) 定义城市坐标。

```
def read_dataset():
    city_coordinates_ = {
        0: (60, 200),
        1: (180, 200),
        2: (80, 180),
        3: (140, 180),
        4: (20, 160),
        5: (100, 160),
        6: (200, 160),
        7: (140, 140),
        8: (40, 120),
        9: (100, 120),
        10: (180, 100),
        11: (60, 80),
        12: (120, 80),
        13: (180, 60),
        14: (20, 40),
        15: (100, 40),
        16: (200, 40),
        17: (20, 20),
        18: (60, 20),
        19: (160, 20),
    }
    return city_coordinates_
```

(3) 距离函数，计算各个城市之间的距离。

```
def get_distance_matrix(city_coordinates: Dict[int, Tuple[int, int]]) -> np.ndarray:
    distance_matrix = np.empty(shape=(num_city, num_city), dtype=np.float_)
    for i in range(num_city):
        xi, yi = city_coordinates[i]
        for j in range(num_city):
            xj, yj = city_coordinates[j]
            distance_matrix[i][j] = np.sqrt(np.power(xi - xj, 2) + np.power(yi - yj, 2))
    return distance_matrix
```

(4) 定义评价函数。

```
def eval_func(x):
```

```

"""
评价函数
:param x: 解
:return: 解的目标函数值
"""
total_distance = 0
for k in range(num_city - 1):
    total_distance += distance_matrix[x[k]][x[k + 1]]
total_distance += distance_matrix[x[-1]][x[0]]
return total_distance

```

(5) 扰动生成新解。

```

def two_opt(x: np.ndarray):
    """
    2-opt swap, 扰动生成新解
    :param x: 解
    :return: 新解
    """
    x = x.copy()
    r1 = np.random.randint(low=0, high=num_city)
    r2 = np.random.randint(low=0, high=num_city)
    x[r1], x[r2] = x[r2], x[r1]
    return x

```

(6) 进行模拟退火过程。

```

def run_simulated_annealing(temp_initial=1e6, temp_final=.1, alpha=.98) -> List:
    """
    :param temp_initial: 初始温度 T0
    :param temp_final: 终止温度 T0
    :param alpha: 降温系数
    :return: 每一代最优解及解的目标函数值
    """
    temp_current = temp_initial # 当前温度

    x = np.random.permutation(num_city) # 初始解, 编码采用常规的整数编码, 如果城市数目为 N,
    那么解就可以表达为 1~N 的随机排列,
    obj_value = eval_func(x) # 初始解目标函数值

    global_best = x # 全局最优解
    trace: List[Tuple[np.ndarray, float]] = [(x, obj_value)] # 记录每一代最优解及解的
    目标函数值
    while temp_current > temp_final: # 外循环: 退火过程

```

```

        for i in range(1000): # 内循环
            obj_value_old = eval_func(x)
            x_ = two_opt(x)
            obj_value_new = eval_func(x_)
            delta_temp = obj_value_new - obj_value_old
            if delta_temp > 0:
                if np.random.uniform(0, 1) < 1. / np.exp(delta_temp / temp_current):
                    x = x_
            else:
                x = x_
                global_best = x
            trace.append((global_best, eval_func(global_best)))
            temp_current *= alpha

    return trace

```

(7) 打印结果。

```

def print_solution(trace: List[Tuple[int, int]]) -> None:
    print(f'城市数量: {num_city}')
    initial_solution, initial_obj_value = trace[0]
    final_solution, final_obj_value = trace[-1]
    print(f'最初路径: {initial_solution}, 最初路径总距离: {initial_obj_value}')
    print(f'最终路径: {final_solution}, 最终路径总距离: {final_obj_value}')

```

(8) 主函数。

```

if __name__ == "__main__":
    city_coordinates = read_dataset()
    num_city = len(city_coordinates)
    distance_matrix: np.ndarray = get_distance_matrix(city_coordinates)
    trace = run_simulated_annealing()
    print_solution(trace)

```

4. 代码运行结果

```

城市数量: 20
最初路径: [ 8 12 18 15 16  0  5 13 11  3 19  9  7  4  6  1 10 17 14  2], 最初路径总距离: 2158.2426596406426
最终路径: [ 1  6 10 13 16 19 12 15 18 17 14 11  8  4  0  2  5  9  7  3], 最终路径总距离: 871.117353844847

```

总 结

本章主要学习和探讨了以下内容：

1. 遗传算法的基本原理和应用，包括：理解基因编码、选择、交叉和变异等基本操作；学会如何根据问题特性设计适应度函数，并实现遗传算法解决具体优化问题（TSP）。
 2. 蚁群算法的基本原理和应用，包括：学习蚂蚁如何通过信息素进行间接通信，并利用这一机制寻找最短路径；实现蚁群算法解决实际优化问题。
 3. 模拟退火算法的基本原理和应用，包括：掌握如何模拟物质加热后再缓慢冷却的过程，并通过概率性地接受解的方式来跳出局部最优，达到全局优化；实现模拟退火算法解决具体的优化问题。
- 通过这三个案例，读者应该系统地掌握了这三种高级优化算法的原理和应用技巧，并能够灵活运用这些算法解决实际复杂优化问题。这些案例不仅帮助读者建立对这些高级算法的直观理解，还使读者在实践中学会算法设计、参数调整以及性能评估等关键知识点的应用。

习 题

1. 使用遗传算法估计函数 $z=100(y-x^2)^2+(1+x)^2$ 在 x 轴范围 $[-2.048, 2.048]$ 、 y 轴范围 $[-2.048, 2.048]$ 的极值。
2. 使用蚁群算法估计函数 $z=100(y-x^2)^2+(1+x)^2$ 在 x 轴范围 $[-2.048, 2.048]$ 、 y 轴范围 $[-2.048, 2.048]$ 的极值。
3. 使用模拟退火算法估计函数 $f(x)=(x-2)(x+3)(x+8)(x-9)$ 在 x 轴范围 $[-10, 10]$ 之间的最小值。

扩展思考

为深化本章知识理解，建议读者通过以下方面进行延伸学习：

1. 混合算法创新：尝试将遗传算法与模拟退火结合，设计新型混合优化策略。例如在遗传算法的变异操作中引入退火机制，观察其对收敛速度的影响。
2. 复杂场景建模：选择城市交通调度或电网优化等实际问题，建立多约束条件的优化模型，测试不同算法的适应能力。
3. 参数敏感型分析：系统研究交叉概率、信息素挥发系数等关键参数对算法性能的影响规律，绘制参数-效果响应曲面。